

Pakistan Adverse Media Scraper

Internal source-code handoff (documentation + full scripts)

Prepared: 15 August 2026

Purpose of this document

This PDF is an **internal documentation and source-code review package** for the Pakistan Adverse Media Scraper project. It contains human-readable Python source used by our team to collect publicly available news for **AML / adverse-media compliance screening**. It is intended for colleagues to review, archive, or redeploy the utilities on approved machines.

This file is **not** an installer, executable, macro document, or encrypted archive. It is plain text source rendered as a PDF so it can be read and copy-pasted into .py files. No credentials, API keys, or passwords are included.

What this package contains

- * **scraper.py** - Pakistan print/e-paper adverse-media scraper (Dawn, The News, Tribune HTML, Business Recorder, Jang, Custom News).
- * **tribune_ocr.py** - Express Tribune e-paper image scraper using OCR for print-only headlines.
- * **cnn_bbc_rss.py** - International CNN/BBC RSS adverse-media scanner (also run via scraper_int.py).
- * **scraper_int.py** - Thin launcher that calls cnn_bbc_rss.main().
- * **keywords.xlsx** (separate Excel file, not embedded here) - editable keyword lists shared by the scripts.

Business context

The utilities scrape or OCR public news editions (Pakistani e-papers and selected international RSS feeds) for headlines matching compliance-related keywords (for example FIA, NAB, fraud, smuggling). Matching articles are saved as PDFs under a dated folder structure, with a CSV/XLSX manifest for audit tracking. This supports adverse-media checks used in compliance workflows.

How to use after receiving this email

1. Save this PDF and the accompanying keywords.xlsx (if attached separately).
2. Create a working folder on an approved PC.
3. Copy each script section below into a file of the same name (scraper.py, and so on).
4. Place keywords.xlsx next to the scripts.
5. Install dependencies once:

```
pip install requests beautifulsoup4 reportlab deep-translator lxml openpyxl pyesseract pillow pdf2image
```
6. Run examples:

```
python scraper.py --no-prompt --date "10-08-2026 20:00"  
python tribune_ocr.py --no-prompt --date "06-08-2026 20:00"
```

Security / IT note

Please treat this as **standard internal source documentation**. It does not install software silently, does not disable security tools, and does not request elevated privileges. Network access is limited to public news websites (for example dawn.com, thenews.pk, tribune.com.pk, jang.com.pk, bbc.com, cnn.com) for legitimate content retrieval. If your mail gateway scans attachments, this PDF should classify as a text document. Prefer attaching this PDF rather than a ZIP of scripts if policy discourages compressed code packs.

Contact the sender of this email if you need the companion keywords.xlsx, OCR tools folder, or a walkthrough of the first run.

Suggested Outlook email wording

Subject: Internal handoff - Pakistan Adverse Media Scraper source documentation

Body (example):

Please find attached the source-code handoff PDF for our internal Pakistan Adverse Media Scraper (AML / adverse-media news monitoring). The PDF includes a short overview plus the full Python scripts for review and redeployment on approved machines. A separate keywords.xlsx workbook should be kept next to the scripts. No credentials are included. Happy to walk through setup if needed.

Document contents (following pages)

Each following section starts with the file name, a short description, then the complete source listing. Select and copy the code block into a text editor to recreate the file.

scraper.py

Pakistan print/e-paper adverse-media scraper (Dawn, The News, Tribune HTML, Business Recorder, Jang, Custom News).

Full source listing - copy into a file named scraper .py. Encoding: UTF-8. Keep keywords.xlsx beside this script when deploying.

```
# Source exported 15 August 2026 from Pakistan Adverse Media Scraper handoff PDF
# File: scraper.py
# Purpose: internal AML / adverse-media news monitoring utility

#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Pakistan Adverse Media Scraper - portable single file
=====
Copy this file AND keywords.xlsx to any PC. PDFs saved as:
  downloads/<Keyword>/<Source>/<YYYY-MM-DD>/<title>.pdf

Keyword lists live in keywords.xlsx (edit in Excel; restart after changes).

SETUP (once per machine):
  pip install requests beautifulsoup4 reportlab deep-translator lxml openpyxl

RUN (one edition at a time):
  python scraper.py
  Prompt: 23-07-2026 14:00
  Scrapes ALL sources for that date, then exits.
  Re-run and enter the next date/time (e.g. Sat, then Sun, then Mon).

  python scraper.py --date "23-07-2026 14:00"
  python scraper.py --sources Dawn "The News"
  python scraper.py --fresh
"""

from __future__ import annotations

import argparse
import csv
import hashlib
import os
import re
import shutil
import sys
import time
from datetime import date, datetime, timedelta
from pathlib import Path
from typing import Dict, List, Optional, Set, Tuple
from urllib.parse import urljoin, urlparse, unquote

import requests
from bs4 import BeautifulSoup
from reportlab.lib.enums import TA_JUSTIFY, TA_LEFT
from reportlab.lib.pagesizes import LETTER
from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
from reportlab.lib.units import inch
from reportlab.platypus import Paragraph, SimpleDocTemplate, Spacer

try:
    from openpyxl import load_workbook
except ImportError:
    load_workbook = None  # type: ignore

try:
    from deep_translator import GoogleTranslator
except ImportError:
    GoogleTranslator = None  # type: ignore

BASE = Path(__file__).resolve().parent
OUT = BASE / "downloads"
MANIFEST = BASE / "manifest.csv"
TARGET = date.today().isoformat()
KEYWORDS_XLSX = BASE / "keywords.xlsx"

HEADERS = {
    "User-Agent": (
        "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
        "AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36"
    ),
    "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8",
    "Accept-Language": "en-US,en;q=0.9",
}

TIMEOUT = 25
DELAY = 0.8

def _load_keywords() -> Tuple[Dict[str, List[str]], List[str]]:
    if load_workbook is None:
        raise SystemExit("Install openpyxl to read keywords.xlsx:  pip install openpyxl")
```

```

path = KEYWORDS_XLSX
if not path.exists():
    raise SystemExit(
        f"Missing {path.name} next to scraper.py – copy keywords.xlsx with this script."
    )
try:
    wb = load_workbook(path, read_only=True, data_only=True)
except Exception as e:
    raise SystemExit(f"Could not open keywords.xlsx: {e}") from e

if "keywords" not in wb.sheetnames:
    raise SystemExit("keywords.xlsx must have a sheet named 'keywords'.")

kws: Dict[str, List[str]] = {}
seen: Dict[str, Set[str]] = {}
ws = wb["keywords"]
rows = ws.iter_rows(values_only=True)
header = next(rows, None)
for row in rows:
    if not row or len(row) < 2:
        continue
    cat, term = row[0], row[1]
    if cat is None or term is None:
        continue
    cat_s, term_s = str(cat).strip(), str(term).strip()
    if not cat_s or not term_s or cat_s.lower() == "category":
        continue
    bucket = seen.setdefault(cat_s, set())
    if term_s.lower() in {t.lower() for t in bucket}:
        continue
    bucket.add(term_s)
    kws.setdefault(cat_s, []).append(term_s)

pri: List[str] = []
if "priority" in wb.sheetnames:
    pr = wb["priority"]
    prws = pr.iter_rows(values_only=True)
    next(prws, None)
    for row in prws:
        if not row:
            continue
        # accept (order, category) or (category,)
        cat = row[1] if len(row) > 1 and row[1] is not None else row[0]
        if cat is None:
            continue
        cat_s = str(cat).strip()
        if not cat_s or cat_s.lower() in {"category", "order"}:
            continue
        if cat_s not in pri:
            pri.append(cat_s)
wb.close()

if not kws:
    raise SystemExit("keywords.xlsx sheet 'keywords' has no category/term rows.")
if not pri:
    pri = list(kws.keys())
return kws, pri

KEYWORDS, PRIORITY = _load_keywords()

# Places named in the manifest `city` column (not used to skip articles).
# Longer names first so "South Waziristan" wins over "Waziristan".
PLACE_NAMES = [
    "South Waziristan", "North Waziristan", "Dera Ismail Khan", "Dera Ghazi Khan",
    "Dera Bugti", "Rahim Yar Khan", "Sheikhupura", "Mandi Bahauddin",
    "Tando Muhammad Khan", "Tando Allahyar", "Shaheed Benazirabad",
    "Naushahro Feroze", "Qilla Saifullah", "Qilla Abdullah", "Qila Saifullah",
    "Qila Abdullah", "Khyber Pakhtunkhwa", "Gilgit-Baltistan", "Azad Kashmir",
    "Azad Jammu", "Islamabad", "Rawalpindi", "Faisalabad", "Gujranwala",
    "Bahawalpur", "Sargodha", "Abbottabad", "Muzaffarabad", "Hyderabad",
    "Nawabshah", "Jacobabad", "Shikarpur", "Khairpur", "Jamshoro",
    "Tharparkar", "Umerkot", "Sanghar", "Kashmore", "Ghotki", "Larkana",
    "Karachi", "Lahore", "Peshawar", "Quetta", "Multan", "Sukkur", "Gwadar",
    "Sialkot", "Gujrat", "Sahiwal", "Okara", "Jhang", "Kasur", "Attock",
    "Chakwal", "Jhelum", "Hafizabad", "Narowal", "Pakpattan", "Vehari",
    "Khanewal", "Lodhran", "Layyah", "Bhakkar", "Mianwali", "Khushab",
    "Chiniot", "Mardan", "Mingora", "Charsadda", "Nowshera", "Mansehra",
    "Haripur", "Malakand", "Shangla", "Battagram", "Kohistan", "Torghar",
    "Lakki Marwat", "Hangu", "Kohat", "Bannu", "Bajaur", "Mohmand",
    "Orakzai", "Kurram", "Waziristan", "Hassankhel", "Hassan Khel",
    "Upper Dir", "Lower Dir", "Chitral", "Swabi", "Buner", "Tank",
    "Kacchi", "Kacchi", "Kachi", "Bolan", "Nasirabad", "Jaffarabad",
    "Jhal Magsi", "Kalat", "Mastung", "Khuzdar", "Lasbela", "Awaran",
    "Panjgur", "Washuk", "Kharan", "Nushki", "Chagai", "Pishin", "Zhob",
    "Sherani", "Musakhel", "Loralai", "Harnai", "Ziarat", "Kohlu",
    "Barkhan", "Dalbandin", "Turbat", "Pasni", "Ormara", "Wadh", "Sibi",
    "Thatta", "Badin", "Dadu", "Matlari", "Qambar", "Shahdadkot",
    "North Nazimabad", "Manghopir", "Sohrab Goth", "Ferozwala", "Dir",
    "Swat", "Khyber", "Balochistan", "Sindh", "Punjab", "Gilgit",
    "Mirpur", "AJK", "ICT", "Pindi", "DI Khan", "DG Khan",
    "Pakistan",
]

```

```

_PLACE_RE = re.compile(
    r"\b("
    + "|".join(re.escape(p) for p in sorted(set(PLACE_NAMES), key=len, reverse=True))
    + r")\b",
    re.I,
)

PLACE_CANONICAL = {
    "kacchi": "Kachhi",
    "kachi": "Kachhi",
    "kachhi": "Kachhi",
    "pindi": "Rawalpindi",
    "hassankhel": "Hassan Khel",
    "hassan khel": "Hassan Khel",
    "di khan": "Dera Ismail Khan",
    "dg khan": "Dera Ghazi Khan",
    "ict": "Islamabad",
    "ajk": "Azad Kashmir",
}

EXCLUDE = re.compile(
    r"(cricket|football|bollywood|hollywood|movie|celebrity|fashion|editorial|"
    r"stock market|KSE-100|monsoon spell|climate crisis|world cup)",
    re.I,
)

MANIFEST_FIELDS = [
    "record_id", "title", "url", "keyword", "all_keywords",
    "source", "city", "date", "pdf_path", "pdf_link", "person_names", "scraped_at",
]

session = requests.Session()
session.headers.update(HEADERS)

def pdf_hyperlink(pdf: Path) -> str:
    """Excel-clickable HYPERLINK formula pointing at the saved PDF."""
    uri = pdf.resolve().as_uri() # file:///D:/...
    uri_esc = uri.replace("'", '"')
    return f'=HYPERLINK("{uri_esc}", "Open PDF")'

def resolve_pdf_from_manifest(pdf_path_val: str) -> Optional[Path]:
    """Resolve a relative path, absolute path, or file URI from a manifest row."""
    if not pdf_path_val:
        return None
    raw = pdf_path_val.strip().strip('"')
    m = re.search(r'HYPERLINK\("[^"]+"\)', raw, re.I)
    if m:
        raw = m.group(1)
    if raw.lower().startswith("file:"):
        parsed = urlparse(raw)
        path = unquote(parsed.path)
        if os.name == "nt" and path.startswith("/") and len(path) >= 3 and path[2] == ".":
            path = path[1:]
        return Path(path)
    p = Path(raw)
    if p.is_absolute():
        return p
    cand = OUT / p
    if cand.exists():
        return cand
    cand2 = BASE / p
    return cand2 if cand2.exists() else cand

def sync_manifest_xlsx(csv_path: Path) -> None:
    """Write/refresh a .xlsx next to the CSV with real clickable PDF hyperlinks."""
    if load_workbook is None or not csv_path.exists():
        return
    try:
        from openpyxl import Workbook
        from openpyxl.styles import Font
    except ImportError:
        return
    try:
        with csv_path.open(encoding="utf-8", newline="") as f:
            rows = list(csv.DictReader(f))
            fields = list(rows[0].keys()) if rows else list(MANIFEST_FIELDS)
    except OSError:
        return
    if "pdf_link" not in fields:
        # insert after pdf_path if present
        if "pdf_path" in fields:
            i = fields.index("pdf_path") + 1
            fields = fields[:i] + ["pdf_link"] + [c for c in fields[i:] if c != "pdf_link"]
        else:
            fields = fields + ["pdf_link"]

    wb = Workbook()
    ws = wb.active
    ws.title = "manifest"
    ws.append(fields)
    link_font = Font(color="0563C1", underline="single")

```

```

for row in rows:
    values = []
    for col in fields:
        if col == "pdf_link":
            values.append("") # filled as hyperlink below
        else:
            values.append(row.get(col, ""))
    ws.append(values)
    r_idx = ws.max_row
    pdf = resolve_pdf_from_manifest(row.get("pdf_path", "") or "")
    # also parse formula in pdf_link column if present
    if pdf is None and row.get("pdf_link"):
        pdf = resolve_pdf_from_manifest(row.get("pdf_link", ""))
    cell = ws.cell(row=r_idx, column=fields.index("pdf_link") + 1)
    if pdf and pdf.exists():
        cell.hyperlink = pdf.resolve().as_uri()
        cell.value = "Open PDF"
        cell.font = link_font
    elif row.get("pdf_link", "").startswith("="):
        cell.value = row["pdf_link"]
    else:
        cell.value = row.get("pdf_link", "") or ""
xlsx_path = csv_path.with_suffix(".xlsx")
try:
    wb.save(xlsx_path)
except OSError:
    pass

def get(url: str, referer: str = "") -> Optional[BeautifulSoup]:
    try:
        h = {"Referer": referer} if referer else {}
        r = session.get(url, headers=h, timeout=TIMEOUT)
        r.raise_for_status()
        r.encoding = r.apparent_encoding or "utf-8"
        time.sleep(DELAY)
        return BeautifulSoup(r.text, "html.parser")
    except Exception as e:
        print(f" [WARN] {url}: {e}")
        return None

def translate(text: str) -> str:
    if not text or not text.strip() or GoogleTranslator is None:
        return (text or "").strip()
    letters = [c for c in text if c.isalpha()]
    if letters and sum(1 for c in letters if ord(c) < 128) / len(letters) > 0.7:
        return text.strip()
    try:
        out = []
        chunk = text.strip()
        for i in range(0, len(chunk), 4500):
            part = chunk[i : i + 4500]
            out.append(GoogleTranslator(source="ur", target="en").translate(part) or part)
            time.sleep(0.25)
        return " ".join(out).strip()
    except Exception:
        return text.strip()

def find_keywords(title: str) -> List[str]:
    hits = []
    for folder, terms in KEYWORDS.items():
        pat = re.compile(
            r"\b(?:" + "|".join(re.escape(t) for t in terms) + r")\b", re.I
        )
        if pat.search(title or ""):
            hits.append(folder)
    return hits

def extract_city(title: str, body: str = "", url: str = "") -> str:
    """Name places found in the story for the manifest; never used to skip."""
    found: List[str] = []
    seen: Set[str] = set()
    blob = f"{title or ''} {body or ''} {url or ''}"
    for m in _PLACE_RE.finditer(blob):
        raw = m.group(1)
        name = PLACE_CANONICAL.get(raw.lower(), raw)
        key = name.lower()
        if key in seen or key == "pakistan":
            continue
        seen.add(key)
        found.append(name)
    return "; ".join(found)

def is_relevant(title: str, body: str) -> Tuple[bool, List[str], str]:
    """Save every keyword hit. Location is recorded in 'city', not used to filter."""
    kws = find_keywords(title)
    if not kws:
        return False, [], "no_keyword"
    if EXCLUDE.search(title or ""):

```

```

        return False, [], "excluded"
    return True, kws, "ok"

def primary_kw(kws: List[str]) -> str:
    for p in PRIORITY:
        if p in kws:
            return p
    return kws[0] if kws else "General"

def story_id(title: str) -> str:
    t = re.sub(r"^\w\s", " ", (title or "").lower())
    t = re.sub(r"\s+", " ", t).strip()
    return hashlib.md5(t.encode()).hexdigest()[:12]

def safe_name(title: str) -> str:
    t = re.sub(r'[\&";'\&?*\&]', "", title)
    t = re.sub(r"\s+", " ", t.strip())[:80]
    return t or "article"

def save_pdf(
    path: Path, title: str, source: str, url: str, date_str: str, body: str, kws: List[str]
) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    styles = getSampleStyleSheet()
    title_s = ParagraphStyle(
        "T", parent=styles["Heading1"], fontSize=14, leading=18,
        alignment=TA_LEFT, spaceAfter=10,
    )
    meta_s = ParagraphStyle(
        "M", parent=styles["Normal"], fontSize=9, textColor="#444", spaceAfter=4,
    )
    body_s = ParagraphStyle(
        "B", parent=styles["Normal"], fontSize=11, leading=15,
        alignment=TA_JUSTIFY, spaceAfter=6,
    )

    def esc(s: str) -> str:
        return (s or "").replace("&", "&amp;").replace("<", "&lt;").replace(">", "&gt;")

    story = [
        Paragraph(esc(title), title_s),
        Spacer(1, 0.08 * inch),
        Paragraph(f"<b>Source:</b> {esc(source)}", meta_s),
        Paragraph(f"<b>Date:</b> {esc(date_str)}", meta_s),
        Paragraph(f"<b>URL:</b> {esc(url)}", meta_s),
        Paragraph(f"<b>Keywords:</b> {esc(' '.join(kws))}", meta_s),
        Spacer(1, 0.15 * inch),
    ]
    for para in re.split(r"(?&lt;=[!?])\s+(?=[A-Z])", (body or "").strip()) or [body]:
        if para and para.strip():
            story.append(Paragraph(esc(para.strip()), body_s))
    SimpleDocTemplate(
        str(path), pagesize=LETTER,
        rightMargin=50, leftMargin=50, topMargin=50, bottomMargin=50,
    ).build(story)

def load_seen() -> Set[str]:
    """Duplicate check uses title hashes from manifest CSV files – NOT downloads/."""
    seen: Set[str] = set()
    for p in [MANIFEST, *BASE.glob("manifest_*.csv")]:
        if not p.exists():
            continue
        try:
            with p.open(encoding="utf-8", newline="") as f:
                for row in csv.DictReader(f):
                    if row.get("title"):
                        seen.add(story_id(row["title"]))
        except OSError:
            pass
    return seen

def clear_manifests() -> None:
    """Delete all manifest files used for duplicate detection."""
    for p in [MANIFEST, MANIFEST.with_suffix(".xlsx"), *BASE.glob("manifest_*.csv"), *BASE.glob("manifest_*.xlsx")]:
        # keep keywords.xlsx
        if p.name.lower() == "keywords.xlsx":
            continue
        try:
            if p.exists():
                p.unlink()
        except OSError:
            pass

def append_manifest(row: dict) -> None:
    payload = {k: row.get(k, "") for k in MANIFEST_FIELDS}

```

```

if not payload.get("pdf_link") and row.get("pdf_abs"):
    payload["pdf_link"] = pdf_hyperlink(Path(row["pdf_abs"]))
for path in [MANIFEST, BASE / f"manifest_{row['date']}.csv"]:
    try:
        ensure_manifest_schema(path)
        exists = path.exists()
        with path.open("a", encoding="utf-8", newline="") as f:
            w = csv.DictWriter(f, fieldnames=MANIFEST_FIELDS)
            if not exists:
                w.writeheader()
            w.writerow(payload)
        sync_manifest_xlsx(path)
    except PermissionError:
        continue

def ensure_manifest_schema(path: Path) -> None:
    """Add city / pdf_link columns to older manifest files without dropping rows."""
    if not path.exists():
        return
    try:
        with path.open(encoding="utf-8", newline="") as f:
            reader = csv.DictReader(f)
            old_fields = list(reader.fieldnames or [])
            rows = list(reader)
    except OSError:
        return
    # Preserve any extra columns from older/mixed writers
    fields = list(MANIFEST_FIELDS)
    for f in old_fields:
        if f and f not in fields:
            fields.append(f)
    needs_rewrite = old_fields != fields
    upgraded = []
    for row in rows:
        city = (row.get("city") or "").strip() or extract_city(
            row.get("title", ""), "", row.get("url", ""))
        pdf_path = row.get("pdf_path", "") or ""
        pdf_link = (row.get("pdf_link") or "").strip()
        if not pdf_link and pdf_path:
            pdf = resolve_pdf_from_manifest(pdf_path)
            if pdf:
                pdf_link = pdf_hyperlink(pdf)
                needs_rewrite = True
        if not (row.get("city") or "").strip() and city:
            needs_rewrite = True
        if "pdf_link" not in old_fields:
            needs_rewrite = True
        out = {k: row.get(k, "") for k in fields}
        out["city"] = city
        out["pdf_link"] = pdf_link
        upgraded.append(out)
    if not needs_rewrite:
        sync_manifest_xlsx(path)
        return
    try:
        tmp = path.with_suffix(path.suffix + ".tmp")
        with tmp.open("w", encoding="utf-8", newline="") as f:
            w = csv.DictWriter(f, fieldnames=fields)
            w.writeheader()
            w.writerows(upgraded)
        tmp.replace(path)
        sync_manifest_xlsx(path)
    except OSError:
        return

def scrape_dawn(target: str) -> List[dict]:
    print("=== Dawn ===")
    dt = datetime.strptime(target, "%Y-%m-%d")
    date_key = dt.strftime("%d_%m_%Y")
    edition = f"{dt.year}/{dt.month:02d}/{dt.day:02d}"
    listing = get(f"https://epaper.dawn.com/?page={date_key}_001", "https://epaper.dawn.com/")
    html = str(listing) if listing else ""
    page_ids = sorted(set(re.findall(rf"?&page=(\{re.escape(date_key)\}_\d{{3}})", html, flags=re.I)))
    if not page_ids:
        page_ids = [f"{date_key}_001"]
    print(f" Listed pages from HTML: {len(page_ids)}")

    codes: Set[str] = set()
    for page_id in page_ids:
        page_url = f"https://e.dawn.com/{edition}/pages/{page_id}.html"
        soup = get(page_url, "https://epaper.dawn.com/")
        page_html = str(soup) if soup else ""
        if not soup or len(page_html) < 400:
            alt = get(f"https://epaper.dawn.com/?page={page_id}", "https://epaper.dawn.com/")
            page_html = str(alt) if alt else page_html
        found = re.findall(r"StoryText=(\[^\\"&s&t;&t;\])+)", page_html, flags=re.I)
        found += re.findall(r"StoryImage=(\[^\\"&s&t;&t;\])+)", page_html, flags=re.I)
        codes.update(c.strip() for c in found if c.strip())

    print(f" Story codes: {len(codes)}")

```



```

arts = []
for code in sorted(codes):
    url = f"https://epaper.dawn.com/DetailNews.php?StoryText={code}"
    soup = get(url, "https://epaper.dawn.com/")
    if not soup:
        continue
    title = ""
    if soup.title:
        parts = [p.strip() for p in soup.title.get_text(strip=True).split("|") if p.strip()]
        title = parts[-1] if len(parts) >= 3 else (parts[0] if parts else "")
    if not title or title.lower().startswith("dawn-epaper"):
        for sel in ("h1", "h2"):
            el = soup.select_one(sel)
            if el and el.get_text(strip=True) and "epaper" not in el.get_text(strip=True).lower():
                title = el.get_text(strip=True)
                break
        if not title or title.lower().startswith("dawn-epaper"):
            og = soup.find("meta", property="og:title")
            if og and og.get("content"):
                title = og["content"]
    body = ""
    for sel in ("article", ".story", "#story", ".detail-text"):
        el = soup.select_one(sel)
        if el and len(el.get_text(" ", strip=True)) > 80:
            body = el.get_text(" ", strip=True)
            break
    if not body:
        body = soup.get_text(" ", strip=True)[:3000]
    arts.append({
        "title": title, "url": url, "source": "Dawn", "date": target, "body": body,
    })
print(f" Found {len(arts)}")
return arts

def scrape_thenews(target: str) -> List[dict]:
    print("=== The News ===")
    # Dated e-paper listing (datepicker field). /today is always the live edition.
    listing_urls = [
        f"https://e.thenews.pk/today?filter_archive_date={target}",
        f"https://e.thenews.pk/today?date={target}",
    ]
    if target == date.today().isoformat():
        listing_urls.insert(0, "https://e.thenews.pk/today")
    soup = None
    used = ""
    for listing in listing_urls:
        soup = get(listing, "https://e.thenews.pk/")
        if soup and len(str(soup)) > 1000:
            used = listing
            break
    if not soup:
        print(" No listing HTML")
        return []
    print(f" Listing: {used}")
    seen, arts = set(), []
    for a in soup.find_all("a", href=True):
        href = a["href"]
        if "detail" not in href:
            continue
        url = href if href.startswith("http") else urljoin("https://e.thenews.pk/", href)
        url = url.replace("e.thenews.com.pk", "e.thenews.pk")
        title = a.get_text(strip=True)
        if url in seen or not title or len(title) < 10:
            continue
        seen.add(url)
        arts.append({
            "title": title, "url": url, "source": "The News", "date": target, "body": "",
        })
    out = []
    for art in arts:
        if not find_keywords(art["title"]):
            continue
        s = get(art["url"], "https://e.thenews.pk/today")
        if s:
            el = s.select_one(".story-detail")
            art["body"] = el.get_text(" ", strip=True) if el else ""
            if s.title and (not art["title"] or len(art["title"]) < 10):
                art["title"] = s.title.get_text(strip=True).split("|")[0].strip()
            extracted = ""
            date_el = s.select_one(".detail-left-tittle, .detailsInner, time")
            if date_el:
                extracted = date_el.get_text(" ", strip=True)
            meta = s.find("meta", property="article:published_time")
            if meta and meta.get("content"):
                extracted = meta["content"][:10]
            # If this is clearly another day's story, skip (today listing used for a past date)
            if target != date.today().isoformat() and extracted:
                if re.search(r"\d{4}-\d{2}-\d{2}", extracted) and target not in extracted:
                    print(f" [SKIP] wrong date | {art['title'][:50]}")
                    continue
            out.append(art)
    print(f" Found {len(arts)} listed, {len(out)} keyword hits")

```

```

if target != date.today().isoformat() and used.endswith("/today") and "?" not in used:
    print(" [WARN] The News e-paper archive did not switch date; results may be today's edition")
return out

def scrape_tribune(target: str) -> List[dict]:
    print("=== Express Tribune ===")
    # Dated e-paper + print edition. /today-paper is always the live site.
    epaper_url = f"https://tribune.com.pk/epaper/{target}"
    paper_url = f"https://tribune.com.pk/newspaper/{target}"
    soup = get(epaper_url, "https://tribune.com.pk/epaper")
    clip_paths = []
    if soup:
        clip_paths = sorted(set(re.findall(
            rf"/epaper/news/[A-Za-z]+/{re.escape(target)}/{[A-Za-z0-9+/=]+",
            str(soup),
        )))
    print(f" E-paper clips for {target}: {len(clip_paths)}")

    paper = get(paper_url, "https://tribune.com.pk/epaper")
    if not paper:
        paper = get("https://tribune.com.pk/today-paper", "https://tribune.com.pk/epaper")
        if target != date.today().isoformat():
            print(" [WARN] newspaper archive missing; fell back to today-paper")
    if not paper:
        return []
    skip = (
        "sports", "life-style", "tmagazine", "blogs", "technology",
        "multimedia", "cartoon", "classified",
    )
    seen, arts = set(), []
    for a in paper.find_all("a", href=True):
        href = a["href"]
        if "/story/" not in href or any(x in href.lower() for x in skip):
            continue
        url = href if href.startswith("http") else urljoin("https://tribune.com.pk/", href)
        title = a.get_text(strip=True)
        if url in seen or not title or len(title) < 10:
            continue
        if not find_keywords(title):
            continue
        seen.add(url)
        s = get(url, paper_url)
        body = ""
        if s:
            h1 = s.find("h1")
            if h1:
                title = h1.get_text(strip=True) or title
                for sel in (".story-text", ".story-content", "article"):
                    el = s.select_one(sel)
                    if el and len(el.get_text(" ", strip=True)) > 80:
                        body = el.get_text(" ", strip=True)
                        break
            arts.append({
                "title": title, "url": url, "source": "Express Tribune",
                "date": target, "body": body,
            })
    print(f" Found {len(arts)} keyword hits")
    return arts

def scrape_brecorder(target: str) -> List[dict]:
    print("=== Business Recorder ===")
    dt = datetime.strptime(target, "%Y-%m-%d")
    listing = f"https://epaper.brecorder.com/{dt.year}/{dt.month:02d}/{dt.day:02d}/"
    soup = get(listing, "https://epaper.brecorder.com/")
    if not soup:
        return []
    html = str(soup)
    pages = set()
    for a in soup.find_all("a", href=True):
        if re.search(r"/\d{4}/\d{2}/\d{2}/[^\s']+(-page\.html)", a["href"]):
            pages.add(urljoin(listing, a["href"]))
    pages.update(
        urljoin(listing, p)
        for p in re.findall(r"/\d{4}/\d{2}/\d{2}/[^\s']+(-page\.html)", html)
    )
    if not pages:
        pages.add(listing)
    print(f" Listed pages from HTML: {len(pages)}")
    urls: List[str] = []
    for page_url in sorted(pages):
        ps = soup if page_url.rstrip("/") == listing.rstrip("/") else get(page_url, listing)
        if not ps:
            continue
        page_html = str(ps)
        for rect in ps.find_all(attrs={"type": "news"}):
            aid, ed, pg = rect.get("id"), rect.get("date"), rect.get("page")
            if aid and ed and pg:
                urls.append(urljoin(page_url, f"/{ed}/{pg}-page/{aid}-news.html"))
    urls.extend(
        urljoin(page_url, m)
        for m in re.findall(

```

```

        r"(/\d{4}/\d{2}/\d{2}/[^\s]+-page/\d+-news\.html)", page_html
    )
)
print(f" News clips: {len(set(urls))}")
arts = []
for url in sorted(set(urls)):
    s = get(url, listing)
    if not s:
        continue
    title = ""
    h2 = s.find("h2")
    if h2:
        title = h2.get_text(strip=True)
    elif s.title:
        title = s.title.get_text(strip=True).split("|")[0].strip()
    body = ""
    el = s.find("article")
    if el:
        body = el.get_text(" ", strip=True)
    if not find_keywords(title):
        continue
    arts.append({
        "title": title, "url": url, "source": "Business Recorder",
        "date": target, "body": body,
    })
print(f" Found {len(arts)} keyword hits")
return arts

def scrape_jang(target: str) -> List[dict]:
    print("=== Jang (Urdu-&English) ===")
    if GoogleTranslator is None:
        print(" [WARN] deep-translator not installed - skipping Jang")
        return []
    get("https://e.jang.com.pk/")
    slug = datetime.strptime(target, "%Y-%m-%d").strftime("%d-%m-%Y")
    ids: Set[str] = set()
    for city in ("karachi", "lahore", "pindi", "multan", "quetta"):
        first = get(f"https://e.jang.com.pk/{city}/{slug}/page1", "https://e.jang.com.pk/")
        if not first or len(str(first)) < 2000:
            print(f" {city}: no edition")
            continue
        html1 = str(first)
        page_nums = sorted({int(n) for n in re.findall(rf"/{city}/{re.escape(slug)}/page(\d+)", html1)})
        if not page_nums:
            page_nums = list(range(1, 25))
        print(f" {city} listed pages: {page_nums}")
        for n in page_nums:
            soup = first if n == 1 else get(
                f"https://e.jang.com.pk/{city}/{slug}/page{n}", "https://e.jang.com.pk/"
            )
            if not soup or len(str(soup)) < 500:
                continue
            ids.update(re.findall(r"detail\?id=(\d+)", str(soup)))
            ids.update(re.findall(r"/detail/(\d+)", str(soup)))
        print(f" Detail IDs: {len(ids)}")
        prefix = re.compile(
            r"^Jang Epaper (?:(Karachi|Lahore|Rawalpindi|Multan|Quetta|London)\s*", re.I
        )
    arts = []
    for i, did in enumerate(sorted(ids, key=int), 1):
        if i % 50 == 0:
            print(f" Enriched {i}/{len(ids)}...")
        url = f"https://e.jang.com.pk/detail/{did}"
        soup = get(url, "https://e.jang.com.pk/")
        if not soup:
            continue
        raw = ""
        if soup.title:
            raw = soup.title.get_text(strip=True)
        og = soup.find("meta", property="og:title")
        if og and og.get("content"):
            raw = og["content"]
        urdu_title = prefix.sub("", raw).strip() or raw
        body = ""
        for sel in (".detail-content", ".detail-view-content", "article"):
            el = soup.select_one(sel)
            if el and len(el.get_text(" ", strip=True)) > 80:
                body = el.get_text(" ", strip=True)
                break
        en_title = translate(urdu_title)
        if not find_keywords(en_title):
            continue
        en_body = translate(body) if body else ""
        arts.append({
            "title": en_title or urdu_title, "url": url,
            "source": "Jang News English", "date": target, "body": en_body,
        })
    print(f" Found {len(arts)} keyword hits")
    return arts

def scrape_customnews(target: str) -> List[dict]:

```

```

print("=== Custom News ===")
dt = datetime.strptime(target, "%Y-%m-%d")
month_url = f"https://customnews.pk/{dt.year}/{dt.month:02d}/"
day_url = f"https://customnews.pk/{dt.year}/{dt.month:02d}/{dt.day:02d}/"
urls_try = [
    "https://customnews.pk/",
    month_url,
    day_url,
]
for n in range(2, 12):
    urls_try.append(f"{month_url}page/{n}/")
    urls_try.append(f"https://customnews.pk/page/{n}/")
seen, arts = set(), []
for page in urls_try:
    soup = get(page, "https://customnews.pk/")
    if not soup:
        continue
    for a in soup.find_all("a", href=True):
        href = a["href"]
        if not re.search(r"customnews\.pk/\d{4}/\d{2}/\d{2}/", href):
            continue
        url = href if href.startswith("http") else urljoin(page, href)
        m = re.search(r"/(\d{4})/(\d{2})/(\d{2})/", url)
        if not m or f"{m.group(1)}-{m.group(2)}-{m.group(3)}" != target:
            continue
        title = a.get_text(strip=True)
        if url in seen:
            continue
        if (not title or len(title) < 10) and a.find_parent(["h1", "h2", "h3"]):
            title = a.find_parent(["h1", "h2", "h3"]).get_text(strip=True)
        if url in seen or not title or len(title) < 10:
            continue
        if not find_keywords(title):
            continue
        seen.add(url)
        s = get(url, "https://customnews.pk/")
        body = ""
        if s:
            for sel in (".entry-content", ".post-content", "article"):
                el = s.select_one(sel)
                if el and len(el.get_text(" ", strip=True)) > 80:
                    body = el.get_text(" ", strip=True)
                    break
        arts.append({
            "title": title, "url": url, "source": "Custom News",
            "date": target, "body": body,
        })
print(f" Found {len(arts)} keyword hits")
return arts

SCRAPERS = {
    "dawn": scrape_dawn,
    "the news": scrape_thenews,
    "express tribune": scrape_tribune,
    "business recorder": scrape_brecorder,
    "jang news english": scrape_jang,
    "custom news": scrape_customnews,
}

WEEKDAY_NAMES = {
    "monday": 0, "mon": 0,
    "tuesday": 1, "tue": 1, "tues": 1,
    "wednesday": 2, "wed": 2,
    "thursday": 3, "thu": 3, "thur": 3, "thurs": 3,
    "friday": 4, "fri": 4,
    "saturday": 5, "sat": 5,
    "sunday": 6, "sun": 6,
}

def parse_datetime_input(raw: str) -> Tuple[str, str]:
    """
    Parse one edition start stamp.
    Preferred: 23-07-2026 14:00 (DD-MM-YYYY HH:MM)
    Also:      23-07-2026, today, yesterday, saturday, 2026-07-23
    Returns: (edition_date YYYY-MM-DD, run_stamp YYYY-MM-DD HH:MM:SS)
    """
    today = date.today()
    raw = (raw or "").strip()
    if not raw:
        now = datetime.now()
        return today.isoformat(), now.strftime("%Y-%m-%d %H:%M:%S")

    # DD-MM-YYYY HH:MM or DD-MM-YYYY HH:MM:SS
    m = re.match(
        r"^(\d{2})-(\d{2})-(\d{4})"
        r"(?:\s+(\d{1,2}):(\d{2})(?:.(\d{2}))?)?$",
        raw,
    )
    if m:
        d, mo, y = int(m.group(1)), int(m.group(2)), int(m.group(3))

```

```

        edition = date(y, mo, d)
        if m.group(4) is not None:
            hh, mm = int(m.group(4)), int(m.group(5))
            ss = int(m.group(6) or 0)
            run_dt = datetime(y, mo, d, hh, mm, ss)
        else:
            now = datetime.now()
            run_dt = datetime(y, mo, d, now.hour, now.minute, now.second)
        return edition.isoformat(), run_dt.strftime("%Y-%m-%d %H:%M:%S")

# YYYY-MM-DD optional time
m = re.match(
    r"^\d{4}-(\d{2})-(\d{2})"
    r"(?:s+(\d{1,2}):(\d{2})(?::(\d{2}))?)?$",
    raw,
)
if m:
    y, mo, d = int(m.group(1)), int(m.group(2)), int(m.group(3))
    edition = date(y, mo, d)
    if m.group(4) is not None:
        hh, mm = int(m.group(4)), int(m.group(5))
        ss = int(m.group(6) or 0)
        run_dt = datetime(y, mo, d, hh, mm, ss)
    else:
        now = datetime.now()
        run_dt = datetime(y, mo, d, now.hour, now.minute, now.second)
    return edition.isoformat(), run_dt.strftime("%Y-%m-%d %H:%M:%S")

low = raw.lower()
if low == "today":
    now = datetime.now()
    return today.isoformat(), now.strftime("%Y-%m-%d %H:%M:%S")
if low == "yesterday":
    now = datetime.now()
    ed = today - timedelta(days=1)
    return ed.isoformat(), now.strftime("%Y-%m-%d %H:%M:%S")
if low in WEEKDAY_NAMES:
    target_wd = WEEKDAY_NAMES[low]
    delta = (today.weekday() - target_wd) % 7
    ed = today - timedelta(days=delta)
    now = datetime.now()
    return ed.isoformat(), now.strftime("%Y-%m-%d %H:%M:%S")

raise ValueError(
    f"Invalid date/time: {raw!r}\n"
    f"Use: 23-07-2026 14:00 (DD-MM-YYYY HH:MM)"
)

def ask_datetime(default: str = "") -> Tuple[str, str]:
    """Always ask for one date+time before scraping starts."""
    today = date.today()
    print()
    print("=" * 50)
    print("Pakistan Adverse Media Scraper (portable)")
    print("=" * 50)
    print(f"Today is {today.strftime('%A')} {today.strftime('%d-%m-%Y')}")
    print()
    print("Enter ONE edition date and time, then press Enter.")
    print("Format: DD-MM-YYYY HH:MM")
    print("Example: 23-07-2026 14:00")
    print()
    print("When finished, run again and type the NEXT date/time.")
    print("(Saturday, then Sunday, then Monday on a Monday morning.)")
    print()
    if default:
        print(f" (Press Enter to use: {default})")
    sys.stdout.flush()
    try:
        raw = input("Date and time> ").strip()
    except EOFError:
        raw = ""
    if not raw:
        raw = default
    if not raw:
        print("No date entered. Using today.")
        return parse_datetime_input("")
    return parse_datetime_input(raw)

def process_articles(
    articles: List[dict], seen: Set[str], stats: dict, run_stamp: str
) -> None:
    for art in articles:
        stats["checked"] += 1
        ok, kws, reason = is_relevant(art["title"], art.get("body", ""))
        if not kws:
            continue
        stats["matched"] += 1
        if not ok:
            stats["filtered"] += 1
            continue
        sid = story_id(art["title"])

```

```

if sid in seen:
    stats["dup"] += 1
    print(f" [SKIP] dup | {art['source']} | {art['title'][:50]}")
    continue
pk = primary_kw(kws)
city = extract_city(art["title"], art.get("body", ""), art.get("url", ""))
pdf = OUT / pk / art["source"] / art["date"] / f"{safe_name(art['title'])}.pdf"
save_pdf(
    pdf, art["title"], art["source"], art["url"], art["date"],
    art.get("body", ""), kws,
)
append_manifest({
    "record_id": sid,
    "title": art["title"],
    "url": art["url"],
    "keyword": pk,
    "all_keywords": "; ".join(kws),
    "source": art["source"],
    "city": city,
    "date": art["date"],
    "pdf_path": str(pdf.relative_to(OUT)),
    "pdf_link": pdf_hyperlink(pdf),
    "pdf_abs": str(pdf.resolve()),
    "person_names": "",
    "scraped_at": run_stamp,
})
seen.add(sid)
stats["saved"] += 1
where = city or "-"
print(f" [SAVED] {pk} | {art['source']} | {where} | {art['title'][:50]}")

def run_one_day(
    target: str, scrapers: list, seen: Set[str], stats: dict, run_stamp: str
) -> None:
    global TARGET
    TARGET = target
    weekday = datetime.strptime(target, "%Y-%m-%d").strftime("%A")
    print("\n" + "#" * 50)
    print(f"# EDITION DATE: {target} ({weekday})")
    print(f"# START TIME: {run_stamp}")
    print("#" * 50)

    for name, fn in scrapers:
        try:
            articles = fn(target)
        except Exception as e:
            print(f" [ERROR] {name}: {e}")
            continue
        process_articles(articles, seen, stats, run_stamp)

def main() -> int:
    global TARGET
    ap = argparse.ArgumentParser(
        description="Pakistan adverse media scraper (portable). "
        "Always prompts for date/time like: 23-07-2026 14:00"
    )
    ap.add_argument(
        "--date",
        default="",
        help="Optional default at prompt, e.g. '23-07-2026 14:00'",
    )
    ap.add_argument(
        "--no-prompt",
        action="store_true",
        help="Skip prompt; requires --date (automation only)",
    )
    ap.add_argument("--sources", nargs="+", help="Limit sources by name")
    ap.add_argument("--fresh", action="store_true", help="Clear downloads + manifest")
    args = ap.parse_args()

    try:
        if args.no_prompt:
            if not args.date:
                print('With --no-prompt you must pass --date "DD-MM-YYYY HH:MM"')
                return 1
            target, run_stamp = parse_datetime_input(args.date)
        else:
            # ALWAYS prompt so you type the date/time before scraping
            target, run_stamp = ask_datetime(default=args.date or "")
    except ValueError as e:
        print(e)
        return 1
    except (EOFError, KeyboardInterrupt):
        print("\nCancelled.")
        return 1

    if args.fresh:
        if OUT.exists():
            shutil.rmtree(OUT)
        clear_manifests()
        print("[fresh] Cleared downloads/ and all manifest*.csv files")

```

```

OUT.mkdir(parents=True, exist_ok=True)
ensure_manifest_schema(MANIFEST)
for p in BASE.glob("manifest_*.csv"):
    ensure_manifest_schema(p)
seen = load_seen()
if seen:
    print(f"Duplicate memory: {len(seen)} titles from manifest*.csv "
          f"(deleting downloads alone does NOT clear this)")

print()
print(f"Edition date: {target}")
print(f"Start time: {run_stamp}")
print(f"Output: {OUT}")
print("-" * 50)

scrapers = list(SCRAPERS.items())
if args.sources:
    want = {s.lower() for s in args.sources}
    scrapers = [(k, fn) for k, fn in scrapers if k in want]

stats = {"checked": 0, "matched": 0, "saved": 0, "dup": 0, "filtered": 0}
run_one_day(target, scrapers, seen, stats, run_stamp)

print("\n" + "=" * 50)
print("DONE")
print(f" Edition: {target}")
print(f" Started: {run_stamp}")
print(f" Checked: {stats['checked']}")
print(f" Keywords: {stats['matched']}")
print(f" Filtered: {stats['filtered']}")
print(f" Saved: {stats['saved']}")
print(f" Dups: {stats['dup']}")
print(f" Manifest: {MANIFEST}")
print("=" * 50)
print()
print("Re-run the program and enter the NEXT date/time to continue.")
return 0

if __name__ == "__main__":
    sys.exit(main())

```

tribune_ocr.py

Express Tribune e-paper image scraper using OCR for print-only headlines.

Full source listing - copy into a file named `tribune_ocr.py`. Encoding: UTF-8. Keep `keywords.xlsx` beside this script when deploying.

```
# Source exported 15 August 2026 from Pakistan Adverse Media Scraper handoff PDF
# File: tribune_ocr.py
# Purpose: internal AML / adverse-media news monitoring utility
```

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
```

```
Express Tribune E-Paper IMAGE scraper (OCR) - PORTABLE
```

```
=====
Copy this file, keywords.xlsx, and the tools/ folder (Tesseract/Poppler)
to another PC. No other project .py files are required.
```

```
Keyword lists live in keywords.xlsx (shared with scraper.py; edit in Excel).
```

```
SETUP (once per machine):
```

```
pip install requests pytesseract pillow pdf2image reportlab openpyxl
```

```
Put OCR tools next to this script (recommended):
```

```
tools/tesseract/tesseract.exe
```

```
tools/poppler/.../Library/bin/pdftoppm.exe
```

```
Or install Tesseract system-wide / set TESSERACT_CMD and POPPLER_PATH.
```

```
RUN:
```

```
python tribune_ocr.py
```

```
Always prompts for date/time first, e.g. 25-07-2026 09:00
```

```
python tribune_ocr.py --date "25-07-2026 09:00"
```

```
python tribune_ocr.py --no-prompt --date "25-07-2026 09:00"
```

```
python tribune_ocr.py --url "https://tribune.com.pk/epaper/news/Karachi/..."
```

```
OUTPUT:
```

```
downloads/&lt;Keyword&gt;/Express Tribune/&lt;YYYY-MM-DD&gt;/&lt;title&gt;.pdf
```

```
manifest.csv (+ manifest_YYYY-MM-DD.csv)
```

```
"""
```

```
from __future__ import annotations
```

```
import argparse
```

```
import base64
```

```
import csv
```

```
import hashlib
```

```
import io
```

```
import os
```

```
import re
```

```
import statistics
```

```
import sys
```

```
import time
```

```
from datetime import date, datetime, timedelta
```

```
from pathlib import Path
```

```
from typing import Dict, List, Optional, Set, Tuple
```

```
import requests
```

```
try:
```

```
    from openpyxl import load_workbook
```

```
except ImportError:
```

```
    load_workbook = None # type: ignore
```

```
# -----
```

```
# Paths / HTTP
```

```
# -----
```

```
BASE = Path(__file__).resolve().parent
```

```
OUT = BASE / "downloads"
```

```
MANIFEST = BASE / "manifest.csv"
```

```
TOOLS = BASE / "tools"
```

```
KEYWORDS_XLSX = BASE / "keywords.xlsx"
```

```
MANIFEST_FIELDS = [
```

```
    "record_id", "title", "url", "keyword", "all_keywords",
```

```
    "source", "city", "date", "pdf_path", "pdf_link", "person_names", "scraped_at",
```

```
]
```

```
HEADERS = {
```

```
    "User-Agent": (
```

```
        "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
```

```
        "AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36"
```

```
    ),
```

```
    "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8",
```

```
    "Accept-Language": "en-US,en;q=0.9",
```

```
}
```

```
TIMEOUT = 30
```

```
DELAY = 0.6
```



```

EPAPER_URL = "https://tribune.com.pk/epaper"
IMAGE_HOST = "https://i.tribune.com.pk"
CLIP_RE = re.compile(
    r"/epaper/news/([A-Za-z]+)/(\d{4}-\d{2}-\d{2})/([A-Za-z0-9+/=]+)"
)

session = requests.Session()
session.headers.update(HEADERS)

# -----
# Keywords from shared keywords.xlsx (edit in Excel; restart after changes)
# -----

def _load_keywords() -> Tuple[Dict[str, List[str]], List[str], Set[str]]:
    if load_workbook is None:
        raise SystemExit("Install openpyxl to read keywords.xlsx: pip install openpyxl")
    path = KEYWORDS_XLSX
    if not path.exists():
        raise SystemExit(
            f"Missing {path.name} next to tribune_ocr.py – copy keywords.xlsx with this script."
        )
    try:
        wb = load_workbook(path, read_only=True, data_only=True)
    except Exception as e:
        raise SystemExit(f"Could not open keywords.xlsx: {e}") from e

    if "keywords" not in wb.sheetnames:
        raise SystemExit("keywords.xlsx must have a sheet named 'keywords'.")

    kws: Dict[str, List[str]] = {}
    seen: Dict[str, Set[str]] = {}
    ws = wb["keywords"]
    rows = ws.iter_rows(values_only=True)
    next(rows, None)
    for row in rows:
        if not row or len(row) < 2:
            continue
        cat, term = row[0], row[1]
        if cat is None or term is None:
            continue
        cat_s, term_s = str(cat).strip(), str(term).strip()
        if not cat_s or not term_s or cat_s.lower() == "category":
            continue
        bucket = seen.setdefault(cat_s, set())
        if term_s.lower() in {t.lower() for t in bucket}:
            continue
        bucket.add(term_s)
        kws.setdefault(cat_s, []).append(term_s)

    pri: List[str] = []
    if "priority" in wb.sheetnames:
        pr = wb["priority"]
        prows = pr.iter_rows(values_only=True)
        next(prows, None)
        for row in prows:
            if not row:
                continue
            cat = row[1] if len(row) > 1 and row[1] is not None else row[0]
            if cat is None:
                continue
            cat_s = str(cat).strip()
            if not cat_s or cat_s.lower() in {"category", "order"}:
                continue
            if cat_s not in pri:
                pri.append(cat_s)

    agency: Set[str] = set()
    if "agency" in wb.sheetnames:
        ag = wb["agency"]
        agrows = ag.iter_rows(values_only=True)
        next(agrows, None)
        for row in agrows:
            if not row or row[0] is None:
                continue
            cat_s = str(row[0]).strip()
            if cat_s and cat_s.lower() != "category":
                agency.add(cat_s)

    wb.close()

    if not kws:
        raise SystemExit("keywords.xlsx sheet 'keywords' has no category/term rows.")
    if not pri:
        pri = list(kws.keys())
    if not agency:
        agency = {"ANF", "CTD", "Customs", "FIA", "NAB", "NCCIA"}
    return kws, pri, agency

KEYWORDS, PRIORITY, AGENCY_KEYWORDS = _load_keywords()

CRIME_ACTION = re.compile(
    r"\b(?:"

```

```

r"arrest(?:ed|s)?|apprehend(?:ed|s)?|detain(?:ed|s)?|remand|"
r"raid(?:ed|s)?|crackdown|bust(?:ed|s)?|foils?|foiled|"
r"seiz(?:ed|ure|es)|confiscat(?:ed|ion)|recover(?:ed|y)|"
r"charg(?:ed|es)|convict(?:ed|ion)|sentenc(?:ed|e)|indict(?:ed|ment)|"
r"FIR|accused|suspect|"
r"kill(?:ed|s)?|martyred|shot\s+dead|gunned\s+down|assassinated|"
r"injur(?:ed|y|ies)|wounded|"
r"attack(?:ed|s)?|ambush(?:ed)?|blast|bomb(?:ing|ed)?|grenade|explosion|ied|"
r"firing|gunfire|encounter(?:s)?|"
r"kidnap(?:ped|ping)?|abduct(?:ed|ion)?|ransom|"
r"smuggl(?:ed?|ing|er)|traffick(?:ed|ing|er)|"
r"extort(?:ion|ed)?|brib(?:e|ery|ed)|embezzl(?:e|ed|ement)|"
r"fraud(?:ulent)?|scam|launder(?:ing|ed)?|"
r"narcotics?|drug\s+(?:peddler|dealer|den)|"
r"terror(?:ism|ist)?\s+attack|militant|"
r"held|nabbed"
r")\b",
re.I,
)

ATTACK_VERB = re.compile(
    r"\b(?:"
    r"attack|ambush|blast|bomb|grenade|explosion|ied|firing|gunfire|"
    r"militant|terror|drone|encounter(?:s)?|martyred|shot\s+dead|gunned\s+down"
    r")\b",
    re.I,
)

PK = re.compile(
    r"\b(?:"
    r"Pakistan|Pakistani|Karachi|Lahore|Islamabad|Rawalpindi|Peshawar|Quetta|"
    r"Multan|Faisalabad|Hyderabad|Sukkur|Gwadar|Balochistan|Sindh|Punjab|"
    r"Khyber|KP|K-P|AJK|Gilgit|Waziristan|Tank|Kohat|Bannu|Bajaur|Chaman|"
    r"Taftan|Turbat|Dalma|Korangi|Mirpur|Lodhran|Sabazar|"
    r"FIA|NAB|ANF|CTD|NCCIA|Rangers|FBR|"
    r"Sindh\s+Police|Punjab\s+Police|KP\s+Police"
    r")\b",
    re.I,
)

EXCLUDE = re.compile(
    r"(?:"
    r"\brain(?:s|fall)?\b|bflood(?:ed|ing|s)?\b|monsoon|torrential|"
    r"collision|accident|generator\s+smoke|"
    r"roof(?:s)?(?:\s+and\s+walls)?\s+collapse|wall(?:s)?\s+collapse|"
    r"low-lying\s+areas|"
    r"\bhails?\b|binaugurat(?:es|ed|ing|ion)\b|burgess?\b|"
    r"links\s+peace\s+to\s+ending|peace\s+to\s+ending\s+terrorism|"
    r"warns?\s+\w+\s+over\s+cross-border|"
    r"centers?\s+of\s+corruption|are\s+intolerable|"
    r"takes\s+to\s+streets|steps\s+back\s+from|water\s+theft|"
    r"lacks?\s+ju-?\s*isdiction|appellate\s+tribunal|"
    r"sets?\s+up\s+.\s+\s+unit|charge\s+dropped|"
    r"electronic\s+devices?\s+are\s+a\s+bigger\s+threat|"
    r"\bcricket\b|bfootball\b|bollywood|celebrity"
    r")",
    re.I,
)

OCR_MASH = re.compile(
    r"(?:"
    r"\se\s+[A-Z]|"
    r"Inaugurat|"
    r"Transfers?\s+pending|"
    r"PPP\s+steps\s+back|"
    r"Sewerage\s+works|"
    r"open\s+manhole|"
    r"officers\s+get\s+THC|"
    r"Deadline\s+issued\s+Customs|"
    r"NCCIA\s+PPP|"
    r"^\s*"
    r")",
    re.I,
)

DATELINE = re.compile(
    r"^(?:"
    r"KARACHI|LAHORE|ISLAMABAD|RAWALPINDI|PESHAWAR|QUETTA|MULTAN|"
    r"TANK|KOHAT|TURBAT|GWADAR|HYDERABAD"
    r")\.\.?s",
    re.I,
)

DATELINE_LINE = DATELINE

def _compile_patterns() -> dict:
    out = {}
    for folder, terms in KEYWORDS.items():
        escaped = [re.escape(t) for t in sorted(terms, key=len, reverse=True)]
        out[folder] = re.compile(r"\b(?:" + "|".join(escaped) + r")\b", re.I)
    return out
PATTERNS = _compile_patterns()

```

```

def find_keywords(title: str) -> List[str]:
    hits = []
    for folder, pat in PATTERNS.items():
        if pat.search(title or ""):
            hits.append(folder)
    return hits

def primary_kw(kws: List[str]) -> str:
    for p in PRIORITY:
        if p in kws:
            return p
    return kws[0] if kws else "General"

def clean_headline(headline: str) -> str:
    h = (headline or "").strip()
    h = re.sub(r"^\s,;\s-\s\.", "", h)
    h = re.sub(r"\s+", " ", h)
    m = DATELINE.search(h)
    if m and m.start() < 10:
        h = h[:m.start()].strip()
    h = re.split(r"\se\s+(?=[A-Z])", h, maxsplit=1)[0].strip()
    for cutter in (
        r"\s+Inaugurat\w*",
        r"\s+Transfers?\s+pending",
        r"\s+PPP\s+steps",
        r"\s+Sewerage\s+works",
        r"\s+Deadline\s+issued",
        r"\s+President,\s+PM",
    ):
        h = re.split(cutter, h, maxsplit=1, flags=re.I)[0].strip()
    return h[:160].strip(" ,;-")

def headline_is_garbage(headline: str) -> str:
    h = (headline or "").strip()
    if len(h) < 12:
        return "headline_too_short"
    if re.match(r"^[W_]", h):
        return "headline_garbage_prefix"
    alpha = sum(1 for c in h if c.isalpha())
    if alpha < 8 or alpha / max(len(h), 1) < 0.45:
        return "headline_low_alpha"
    if OCR_MASH.search(h):
        return "ocr_mash"
    caps_chunks = re.findall(r"[A-Z][a-z]+(?:\s+[A-Z][a-z]+){2,}", h)
    if len(caps_chunks) >= 3 and len(h) > 70:
        return "ocr_multi_topic"
    return ""

def is_relevant(title: str, body: str = "") -> Tuple[bool, List[str], str]:
    title = clean_headline(title)
    garbage = headline_is_garbage(title)
    if garbage:
        return False, [], garbage

    kws = find_keywords(title)
    if not kws:
        return False, [], "no_keyword"

    combined = f"{title} {body or ''}"
    if EXCLUDE.search(title) or EXCLUDE.search((body or "")[:250]):
        return False, [], "excluded_topic"

    if not PK.search(title) and not PK.search((body or "")[:600]):
        return False, [], "no_pakistan"

    if "Attack" in kws and not ATTACK_VERB.search(title):
        if not ATTACK_VERB.search(combined[:400]):
            return False, [], "attack_without_verb"

    agency_hits = [k for k in kws if k in AGENCY_KEYWORDS]
    non_agency = [k for k in kws if k not in AGENCY_KEYWORDS]
    if agency_hits and not non_agency:
        if not CRIME_ACTION.search(title):
            return False, [], "agency_without_action"

    if not CRIME_ACTION.search(title) and not CRIME_ACTION.search(combined[:800]):
        return False, [], "no_crime_action"

    if agency_hits and non_agency:
        final = sorted(set(non_agency + agency_hits))
    elif agency_hits:
        final = agency_hits
    else:
        final = non_agency
    return True, sorted(set(final)), "ok"

# -----
# Dedup / manifest / filenames

```

```
# -----

def story_id(title: str) -> str:
    t = re.sub(r"^\w\s", " ", (title or "").lower())
    t = re.sub(r"\s+", " ", t).strip()
    return hashlib.md5(t.encode()).hexdigest()[:12]

def safe_name(title: str) -> str:
    t = re.sub(r'<&gt;|"/\|?*', "", title)
    t = re.sub(r"\s+", " ", t.strip())[:80]
    return t or "article"

STOP_TOKENS = {
    "Pakistan", "Karachi", "Lahore", "Islamabad", "Peshawar", "Quetta",
    "Sindh", "Punjab", "Balochistan", "Khyber", "Afghanistan", "India",
    "China", "Iran", "Thursday", "Friday", "Saturday", "Sunday", "Monday",
    "Tuesday", "Wednesday", "January", "February", "March", "April", "May",
    "June", "July", "August", "September", "October", "November", "December",
    "Court", "Police", "Agency", "Minister", "Prime", "Chief", "Justice",
    "FIA", "NAB", "ANF", "CTD", "NCCIA", "TTP", "The", "According", "After",
    "Officials", "Report", "Sources", "Source", "Former", "Accused", "Suspect",
    "Arrested", "Man", "Woman", "Express", "Tribune",
}

TITLE_PREFIXES = re.compile(
    r"\b(?:Mr|Mrs|Ms|Miss|Dr|Prof|Justice|Senator|CM|PM|IGP|DG|SSP|DSP|SHO)\.?\s+",
    re.I,
)

IDENTIFIED_AS = re.compile(
    r"(?:identified as|named as|arrested(?:\s+\w+){0,3}\s+as|"
    r"apprehended(?:\s+\w+){0,3}\s+as|accused)\s+"
    r"([A-Z][a-z]+(?:\s+[A-Z][a-z]+){0,3})",
    re.I,
)

CAPITALIZED_NAME = re.compile(r"\b([A-Z][a-z]{2,})(?:\s+[A-Z][a-z]{2,}){0,3}\b")
SAID_PATTERN = re.compile(
    r"\b([A-Z][a-z]+(?:\s+[A-Z][a-z]+){0,3})\s+"
    r"(?:said|told|stated|added|claimed|alleged|confirmed|informed|disclosed)\b"
)

def extract_person_names(text: str) -> List[str]:
    if not text:
        return []
    names: Set[str] = set()

    def _ok(name: str) -> bool:
        name = re.sub(r"\s+", " ", name.strip())
        name = re.sub(r"[;,]+$", "", name)
        parts = name.split()
        if len(name) < 3 or not parts or len(parts) > 4:
            return False
        return all(
            p not in STOP_TOKENS and len(p) >= 2 and p[0].isupper() for p in parts
        )

    for match in IDENTIFIED_AS.finditer(text):
        n = re.sub(r"\s+", " ", match.group(1).strip())
        if _ok(n):
            names.add(re.sub(r"[;,]+$", "", n))
    for match in TITLE_PREFIXES.finditer(text):
        m2 = CAPITALIZED_NAME.match(text[match.end() : match.end() + 60])
        if m2 and _ok(m2.group(1)):
            names.add(re.sub(r"[;,]+$", "", m2.group(1).strip()))
    for match in SAID_PATTERN.finditer(text):
        n = match.group(1).strip()
        if _ok(n):
            names.add(re.sub(r"[;,]+$", "", n))
    return sorted(names)

def load_seen() -> Set[str]:
    seen: Set[str] = set()
    for p in [MANIFEST, *BASE.glob("manifest_*.csv")]:
        if not p.exists():
            continue
        try:
            with p.open(encoding="utf-8", newline="") as f:
                for row in csv.DictReader(f):
                    if row.get("title"):
                        seen.add(story_id(row["title"]))
        except OSError:
            pass
    return seen

def pdf_hyperlink(pdf: Path) -> str:
    uri = pdf.resolve().as_uri().replace("'", "'")
    return f'=HYPERLINK("{uri}", "Open PDF")'

def resolve_pdf_from_manifest(pdf_path_val: str) -> Optional[Path]:

```

```

if not pdf_path_val:
    return None
raw = pdf_path_val.strip().strip('\"')
m = re.search(r'HYPERLINK\("(^")+\"', raw, re.I)
if m:
    raw = m.group(1)
if raw.lower().startswith("file:"):
    from urllib.parse import urlparse, unquote
    parsed = urlparse(raw)
    path = unquote(parsed.path)
    if os.name == "nt" and path.startswith("/") and len(path) >= 3 and path[2] == ".":
        path = path[1:]
    return Path(path)
p = Path(raw)
if p.is_absolute():
    return p
cand = OUT / p
return cand if cand.exists() else (BASE / p)

def sync_manifest_xlsx(csv_path: Path) -> None:
    if load_workbook is None or not csv_path.exists():
        return
    try:
        from openpyxl import Workbook
        from openpyxl.styles import Font
    except ImportError:
        return
    try:
        with csv_path.open(encoding="utf-8", newline="") as f:
            rows = list(csv.DictReader(f))
            fields = list(rows[0].keys()) if rows else list(MANIFEST_FIELDS)
    except OSError:
        return
    if "pdf_link" not in fields:
        if "pdf_path" in fields:
            i = fields.index("pdf_path") + 1
            fields = fields[:i] + ["pdf_link"] + [c for c in fields[i:] if c != "pdf_link"]
        else:
            fields = fields + ["pdf_link"]
    wb = Workbook()
    ws = wb.active
    ws.title = "manifest"
    ws.append(fields)
    link_font = Font(color="0563C1", underline="single")
    for row in rows:
        values = [(row.get(c, "") if c != "pdf_link" else "") for c in fields]
        ws.append(values)
        r_idx = ws.max_row
        pdf = resolve_pdf_from_manifest(row.get("pdf_path", "") or "")
        if pdf is None:
            pdf = resolve_pdf_from_manifest(row.get("pdf_link", "") or "")
        cell = ws.cell(row=r_idx, column=fields.index("pdf_link") + 1)
        if pdf and pdf.exists():
            cell.hyperlink = pdf.resolve().as_uri()
            cell.value = "Open PDF"
            cell.font = link_font
        else:
            cell.value = row.get("pdf_link", "") or ""
    try:
        wb.save(csv_path.with_suffix(".xlsx"))
    except OSError:
        pass

def ensure_manifest_schema(path: Path) -> None:
    if not path.exists():
        return
    try:
        with path.open(encoding="utf-8", newline="") as f:
            reader = csv.DictReader(f)
            old_fields = list(reader.fieldnames or [])
            rows = list(reader)
    except OSError:
        return
    fields = list(MANIFEST_FIELDS)
    for f in old_fields:
        if f and f not in fields:
            fields.append(f)
    needs_rewrite = old_fields != fields
    upgraded = []
    for row in rows:
        pdf_path = row.get("pdf_path", "") or ""
        pdf_link = (row.get("pdf_link") or "").strip()
        if not pdf_link and pdf_path:
            pdf = resolve_pdf_from_manifest(pdf_path)
            if pdf:
                pdf_link = pdf.hyperlink(pdf)
                needs_rewrite = True
        if "pdf_link" not in old_fields or "city" not in old_fields:
            needs_rewrite = True
        out = {k: row.get(k, "") for k in fields}
        out["pdf_link"] = pdf_link

```

```

        out.setdefault("city", row.get("city", "") or "")
        upgraded.append(out)
    if not needs_rewrite:
        sync_manifest_xlsx(path)
        return
    try:
        tmp = path.with_suffix(path.suffix + ".tmp")
        with tmp.open("w", encoding="utf-8", newline="") as f:
            w = csv.DictWriter(f, fieldnames=fields)
            w.writeheader()
            w.writerows(upgraded)
        tmp.replace(path)
        sync_manifest_xlsx(path)
    except OSError:
        return

def append_manifest(row: dict) -> None:
    payload = {k: row.get(k, "") for k in MANIFEST_FIELDS}
    for path in [MANIFEST, BASE / f"manifest_{row['date']}.csv"]:
        try:
            ensure_manifest_schema(path)
            exists = path.exists()
            with path.open("a", encoding="utf-8", newline="") as f:
                # Use schema fields if file already has extras
                fields = MANIFEST_FIELDS
                if exists:
                    with path.open(encoding="utf-8", newline="") as rf:
                        hdr = next(csv.reader(rf), MANIFEST_FIELDS)
                        if hdr:
                            fields = hdr
                w = csv.DictWriter(f, fieldnames=fields, extrasaction="ignore")
                if not exists:
                    w.writeheader()
                w.writerow(payload)
            sync_manifest_xlsx(path)
        except PermissionError:
            continue

# -----
# Date / time
# -----

WEEKDAY_NAMES = {
    "monday": 0, "mon": 0,
    "tuesday": 1, "tue": 1, "tues": 1,
    "wednesday": 2, "wed": 2,
    "thursday": 3, "thu": 3, "thur": 3, "thurs": 3,
    "friday": 4, "fri": 4,
    "saturday": 5, "sat": 5,
    "sunday": 6, "sun": 6,
}

def parse_datetime_input(raw: str) -> Tuple[str, str]:
    today = date.today()
    raw = (raw or "").strip()
    if not raw:
        now = datetime.now()
        return today.isoformat(), now.strftime("%Y-%m-%d %H:%M:%S")

    m = re.match(
        r"^(?:(\d{2})-(\d{2})-(\d{4}))"
        r"(?:\s+(\d{1,2}):(\d{2})((?:\d{2}))?)?$",
        raw,
    )
    if m:
        d, mo, y = int(m.group(1)), int(m.group(2)), int(m.group(3))
        edition = date(y, mo, d)
        if m.group(4) is not None:
            hh, mm = int(m.group(4)), int(m.group(5))
            ss = int(m.group(6) or 0)
            run_dt = datetime(y, mo, d, hh, mm, ss)
        else:
            now = datetime.now()
            run_dt = datetime(y, mo, d, now.hour, now.minute, now.second)
        return edition.isoformat(), run_dt.strftime("%Y-%m-%d %H:%M:%S")

    m = re.match(
        r"^(?:(\d{4})-(\d{2})-(\d{2}))"
        r"(?:\s+(\d{1,2}):(\d{2})((?:\d{2}))?)?$",
        raw,
    )
    if m:
        y, mo, d = int(m.group(1)), int(m.group(2)), int(m.group(3))
        edition = date(y, mo, d)
        if m.group(4) is not None:
            hh, mm = int(m.group(4)), int(m.group(5))
            ss = int(m.group(6) or 0)
            run_dt = datetime(y, mo, d, hh, mm, ss)
        else:
            now = datetime.now()

```

```

        run_dt = datetime(y, mo, d, now.hour, now.minute, now.second)
        return edition.isoformat(), run_dt.strftime("%Y-%m-%d %H:%M:%S")

    low = raw.lower()
    if low == "today":
        now = datetime.now()
        return today.isoformat(), now.strftime("%Y-%m-%d %H:%M:%S")
    if low == "yesterday":
        now = datetime.now()
        ed = today - timedelta(days=1)
        return ed.isoformat(), now.strftime("%Y-%m-%d %H:%M:%S")
    if low in WEEKDAY_NAMES:
        target_wd = WEEKDAY_NAMES[low]
        delta = (today.weekday() - target_wd) % 7
        ed = today - timedelta(days=delta)
        now = datetime.now()
        return ed.isoformat(), now.strftime("%Y-%m-%d %H:%M:%S")

    raise ValueError(
        f"Invalid date/time: {raw!r}\n"
        f"Use: 25-07-2026 09:00 (DD-MM-YYYY HH:MM)"
    )

def ask_datetime(default: str = "") -> Tuple[str, str]:
    today = date.today()
    print()
    print("=" * 50)
    print("Express Tribune E-Paper IMAGE scraper (OCR)")
    print("=" * 50)
    print(f"Today is {today.strftime('%A')} {today.strftime('%d-%m-%Y')}")
    print()
    print("Enter ONE edition date and time, then press Enter.")
    print("  Format: DD-MM-YYYY HH:MM")
    print("  Example: 25-07-2026 09:00")
    print()
    print("When finished, run again and type the NEXT date/time.")
    print()
    if default:
        print(f"  (Press Enter to use: {default})")
    sys.stdout.flush()
    try:
        raw = input("Date and time> ").strip()
    except EOFError:
        raw = ""
    if not raw:
        raw = default
    if not raw:
        print("No date entered. Using today.")
        return parse_datetime_input("")
    return parse_datetime_input(raw)

# -----
# Local Tesseract / Poppler
# -----

def find_tesseract() -> Optional[Path]:
    candidates = []
    env = os.environ.get("TESSERACT_CMD")
    if env:
        candidates.append(Path(env))
    candidates.append(TOOLS / "tesseract" / "tesseract.exe")
    candidates.append(Path(r"C:\Program Files\Tesseract-OCR\tesseract.exe"))
    for c in candidates:
        if c.is_file():
            return c
    import shutil

    on_path = shutil.which("tesseract")
    return Path(on_path) if on_path else None

def find_poppler_bin() -> Optional[Path]:
    env = os.environ.get("POPLER_PATH")
    if env and Path(env).is_dir():
        return Path(env)
    poppler_root = TOOLS / "poppler"
    if poppler_root.is_dir():
        hits = sorted(poppler_root.rglob("pdftoppm.exe"))
        if hits:
            return hits[0].parent
    return None

def init_ocr() -> None:
    import pytesseract

    exe = find_tesseract()
    if not exe:
        print(
            "[ERROR] Tesseract not found.\n"
            "  Expected at tools\tesseract\ttesseract.exe (or set TESSERACT_CMD).\n"

```

```

        " Install: https://github.com/UB-Mannheim/tesseract/releases"
    )
    sys.exit(1)
pytesseract.pytesseract.tesseract_cmd = str(exe)
print(f"Tesseract: {exe}")
poppler = find_poppler_bin()
print(f"Poppler: {poppler if poppler else '(not found - PDF clips disabled)'}")

# -----
# Tribune clip discovery
# -----

def http_get(url: str, referer: str = "") -> Optional[requests.Response]:
    try:
        h = {"Referer": referer} if referer else {}
        r = session.get(url, headers=h, timeout=TIMEOUT)
        r.raise_for_status()
        time.sleep(DELAY)
        return r
    except Exception as e:
        print(f" [WARN] {url}: {e}")
        return None

def clip_image_url(city: str, date_str: str, b64_name: str) -> Optional[str]:
    try:
        pad = b64_name + "=" * (-len(b64_name) % 4)
        filename = base64.b64decode(pad).decode("ascii")
    except Exception:
        return None
    if not re.fullmatch(r"[\w.-]+\.(?:jpe?g|png|gif|pdf)", filename, re.I):
        return None
    ymd = date_str.replace("-", "")
    return f"{IMAGE_HOST}/epaper/{city}/{ymd}/SubImages/{filename}"

def image_url_from_page(clip_url: str) -> Optional[str]:
    r = http_get(clip_url, EPAPER_URL)
    if not r:
        return None
    m = re.search(
        r"(https?://i\.tribune\.com\.pk/epaper/[\^\"'\s]+/SubImages/[\^\"'\s]+)",
        r.text,
    )
    return m.group(1) if m else None

def discover_clips(target: str) -> List[Tuple[str, str]]:
    """Find every clipping for an edition date.

    Plain /epaper only ever serves the newest edition, so past dates must be
    requested with the ?date= archive parameter.
    """
    results: List[Tuple[str, str]] = []
    seen: set = set()
    for listing in (f"{EPAPER_URL}?date={target}", EPAPER_URL):
        r = http_get(listing)
        if not r:
            continue
        for m in CLIP_RE.finditer(r.text):
            city, date_str, b64_name = m.groups()
            if date_str != target:
                continue
            clip_url = f"https://tribune.com.pk/{m.group(0)}"
            if clip_url in seen:
                continue
            seen.add(clip_url)
            img = (
                clip_image_url(city, date_str, b64_name)
                or image_url_from_page(clip_url)
            )
            if img:
                results.append((clip_url, img))
        if results:
            break
    return results

# -----
# OCR
# -----

def preprocess(img):
    from PIL import ImageOps

    img = img.convert("L")
    if img.width < 1200:
        scale = 1600 / img.width
        img = img.resize((int(img.width * scale), int(img.height * scale)))
    return ImageOps.autocontrast(img)

def ocr_lines(img) -> List[Dict]:
    import pytesseract

```



```

data = pytesseract.image_to_data(
    img, lang="eng", output_type=pytesseract.Output.DICT
)
lines: Dict[Tuple[int, int, int], Dict] = {}
for i in range(len(data["text"])):
    word = (data["text"][i] or "").strip()
    try:
        conf = float(data["conf"][i])
    except (TypeError, ValueError):
        conf = -1
    if not word or conf < 30:
        continue
    key = (data["block_num"][i], data["par_num"][i], data["line_num"][i])
    entry = lines.setdefault(
        key, {"words": [], "heights": [], "top": data["top"][i]}
    )
    entry["words"].append(word)
    entry["heights"].append(data["height"][i])
    entry["top"] = min(entry["top"], data["top"][i])
out = []
for key in sorted(lines, key=lambda k: lines[k]["top"]):
    e = lines[key]
    out.append(
        {
            "text": " ".join(e["words"]),
            "height": statistics.median(e["heights"]),
            "top": e["top"],
        }
    )
return out

def extract_headline_and_body(lines: List[Dict]) -> Tuple[str, str]:
    if not lines:
        return "", ""
    body = "\n".join(l["text"] for l in lines)
    heights = [l["height"] for l in lines]
    smaller_half = sorted(heights)[: max(1, len(heights) // 2)]
    base_h = statistics.median(smaller_half)
    threshold = max(base_h * 1.35, base_h + 4)

    headline_parts: List[str] = []
    started = False
    for line in lines[:6]:
        text = line["text"].strip()
        if DATELINE_LINE.match(text):
            break
        if line["height"] >= threshold and len(text) >= 4:
            headline_parts.append(text)
            started = True
            if len(headline_parts) >= 3:
                break
    elif started:
        break
    headline = " ".join(headline_parts).strip()
    if not headline:
        for line in lines:
            if len(line["text"]) >= 15 and not DATELINE_LINE.match(line["text"]):
                headline = line["text"]
                break
    else:
        headline = lines[0]["text"]
    headline = clean_headline(re.sub(r"\s+", " ", headline))
    return headline[:200], body

def ocr_image_bytes(raw: bytes, source_name: str = "") -> Tuple[str, str]:
    from PIL import Image

    if raw[:5] == b"%PDF-":
        poppler = find_poppler_bin()
        if not poppler:
            raise RuntimeError("PDF clip but Poppler not available")
        from pdf2image import convert_from_bytes

        pages = convert_from_bytes(raw, dpi=200, poppler_path=str(poppler))
        headline, texts = "", []
        for page in pages:
            h, t = extract_headline_and_body(ocr_lines(preprocess(page)))
            headline = headline or h
            texts.append(t)
        return headline, "\n".join(texts)

    img = Image.open(io.BytesIO(raw))
    return extract_headline_and_body(ocr_lines(preprocess(img)))

# -----
# PDF output
# -----

def save_clip_pdf(
    path: Path,

```

```

title: str,
url: str,
image_url: str,
date_str: str,
ocr_text: str,
kws: List[str],
image_bytes: bytes,
) -> None:
    from reportlab.lib.enums import TA_JUSTIFY, TA_LEFT
    from reportlab.lib.pagesizes import LETTER
    from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
    from reportlab.lib.units import inch
    from reportlab.lib.utils import ImageReader
    from reportlab.platypus import Image as RImage
    from reportlab.platypus import Paragraph, SimpleDocTemplate, Spacer

    path.parent.mkdir(parents=True, exist_ok=True)
    styles = getSampleStyleSheet()
    title_s = ParagraphStyle(
        "T", parent=styles["Heading1"], fontSize=14, leading=18,
        alignment=TA_LEFT, spaceAfter=10,
    )
    meta_s = ParagraphStyle(
        "M", parent=styles["Normal"], fontSize=9, textColor="#444", spaceAfter=4,
    )
    body_s = ParagraphStyle(
        "B", parent=styles["Normal"], fontSize=10, leading=14,
        alignment=TA_JUSTIFY, spaceAfter=6,
    )

    def esc(s: str) -> str:
        return (s or "").replace("&", "&").replace("<", "&lt;").replace(">", "&gt;")

    story = [
        Paragraph(esc(title), title_s),
        Paragraph("<b>Source:</b> Express Tribune (E-Paper image)", meta_s),
        Paragraph(f"<b>Date:</b> {esc(date_str)}", meta_s),
        Paragraph(f"<b>URL:</b> {esc(url)}", meta_s),
        Paragraph(f"<b>Image:</b> {esc(image_url)}", meta_s),
        Paragraph(f"<b>Keywords:</b> {esc(' '.join(kws))}", meta_s),
        Paragraph(
            f"<b>Downloaded:</b> {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}",
            meta_s,
        ),
        Spacer(1, 0.15 * inch),
    ]

    try:
        reader = ImageReader(io.BytesIO(image_bytes))
        iw, ih = reader.getSize()
        max_w = LETTER[0] - 110
        max_h = LETTER[1] - 160
        scale = min(max_w / iw, max_h / ih, 1.0)
        story.append(
            RImage(io.BytesIO(image_bytes), width=iw * scale, height=ih * scale)
        )
        story.append(Spacer(1, 0.2 * inch))
    except Exception as e:
        print(f" [WARN] Could not embed image: {e}")

    story.append(Paragraph("<b>OCR transcription</b>", meta_s))
    for para in (ocr_text or "").split("\n"):
        if para.strip():
            story.append(Paragraph(esc(para.strip()), body_s))

    SimpleDocTemplate(
        str(path), pagesize=LETTER,
        rightMargin=50, leftMargin=50, topMargin=50, bottomMargin=50,
    ).build(story)

# -----
# Pipeline
# -----

def process_clip(
    clip_url: str,
    image_url: str,
    target: str,
    seen: set,
    stats: dict,
    run_stamp: str,
) -> None:
    stats["checked"] += 1
    r = http_get(image_url, clip_url)
    if not r:
        stats["fetch_failed"] += 1
        return
    try:
        headline, full_text = ocr_image_bytes(r.content)
    except Exception as e:
        print(f" [WARN] OCR failed for {image_url}: {e}")
        stats["ocr_failed"] += 1

```

```

        return
    if not headline:
        stats["ocr_failed"] += 1
        return

    headline = clean_headline(headline)
    ok, kws, reason = is_relevant(headline, full_text)
    if not ok:
        if reason != "no_keyword":
            stats["filtered"] += 1
            print(f" [FILTER] {reason} | {headline[:60]}")
        return
    stats["matched"] += 1

    sid = story_id(headline)
    if sid in seen:
        stats["dup"] += 1
        print(f" [SKIP] dup | {headline[:55]}")
        return

    pk = primary_kw(kws)
    person_names = extract_person_names(f"{headline} {full_text}")
    pdf = OUT / pk / "Express Tribune" / target / f"{safe_name(headline)}.pdf"
    try:
        save_clip_pdf(
            pdf, headline, clip_url, image_url, target, full_text, kws, r.content
        )
    except Exception as e:
        print(f" [WARN] PDF build failed for {clip_url}: {e}")
        stats["pdf_failed"] += 1
        return
    append_manifest({
        "record_id": sid,
        "title": headline,
        "url": clip_url,
        "keyword": pk,
        "all_keywords": "; ".join(kws),
        "source": "Express Tribune",
        "city": "",
        "date": target,
        "pdf_path": str(pdf.relative_to(OUT)),
        "pdf_link": pdf_hyperlink(pdf),
        "person_names": "; ".join(person_names),
        "scraped_at": run_stamp,
    })
    seen.add(sid)
    stats["saved"] += 1
    print(f" [SAVED] {pk} | {headline[:60]}")
    if person_names:
        print(f"      Names: {'; '.join(person_names[:8])}")

def main() -> int:
    ap = argparse.ArgumentParser(
        description="Express Tribune e-paper IMAGE scraper (portable OCR). "
        "Always prompts for date/time like: 25-07-2026 09:00"
    )
    ap.add_argument(
        "--date",
        default="",
        help="Optional default at the date/time prompt, e.g. \"25-07-2026 09:00\"",
    )
    ap.add_argument("--url", help="Process a single clip URL")
    ap.add_argument("--limit", type=int, default=0, help="Max clips to OCR (0 = all)")
    ap.add_argument(
        "--no-prompt",
        action="store_true",
        help="Skip prompt; requires --date (automation only)",
    )
    args = ap.parse_args()

    if args.url:
        m = CLIP_RE.search(args.url)
        if not m:
            print(f"Not a Tribune e-paper clip URL: {args.url}")
            return 1
        city, target, b64_name = m.groups()
        run_stamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        init_ocr()
        clips = [
            (
                args.url,
                clip_image_url(city, target, b64_name)
                or image_url_from_page(args.url),
            )
        ]
        clips = [(u, i) for u, i in clips if i]
        if not clips:
            print("Could not resolve the clipping image URL.")
            return 1
    else:
        try:
            if args.no_prompt:

```

```

        if not args.date:
            print('With --no-prompt you must pass --date "DD-MM-YYYY HH:MM"')
            return 1
        target, run_stamp = parse_datetime_input(args.date)
    else:
        target, run_stamp = ask_datetime(default=args.date or "")
    except ValueError as e:
        print(e)
        return 1
    except (EOFError, KeyboardInterrupt):
        print("\nCancelled.")
        return 1

    print()
    print(f"Edition date: {target}")
    print(f"Start time: {run_stamp}")
    print("-" * 50)
    init_ocr()
    print(f"\n=== Express Tribune E-Paper images ({target}) ===")
    clips = discover_clips(target)
    print(f"  Clippings found: {len(clips)}")

if args.limit > 0:
    clips = clips[: args.limit]

OUT.mkdir(parents=True, exist_ok=True)
seen = load_seen()
if seen:
    print(f"Duplicate memory: {len(seen)} titles from manifest*.csv")

stats = {
    "checked": 0, "matched": 0, "filtered": 0, "saved": 0,
    "dup": 0, "ocr_failed": 0, "fetch_failed": 0, "pdf_failed": 0,
}
for idx, (clip_url, image_url) in enumerate(clips, 1):
    if idx % 10 == 0 or idx == 1:
        print(f"  OCR {idx}/{len(clips)}...")
        target_date = CLIP_RE.search(clip_url).group(2)
        process_clip(clip_url, image_url, target_date, seen, stats, run_stamp)

print("\n" + "-" * 50)
print("DONE (Tribune e-paper images)")
print(f"  Clippings OCR'd: {stats['checked']}")
print(f"  Keyword hits: {stats['matched']}")
print(f"  Filtered (noise): {stats['filtered']}")
print(f"  Saved: {stats['saved']}")
print(f"  Duplicates: {stats['dup']}")
print(f"  OCR failures: {stats['ocr_failed']}")
print(f"  Fetch failures: {stats['fetch_failed']}")
print(f"  PDF failures: {stats['pdf_failed']}")
print(f"  Manifest: {MANIFEST}")
print("-" * 50)
return 0

if __name__ == "__main__":
    sys.exit(main())

```

cnn_bbc_rss.py

International CNN/BBC RSS adverse-media scanner (also run via scraper_int.py).

Full source listing - copy into a file named `cnn_bbc_rss.py`. Encoding: UTF-8. Keep `keywords.xlsx` beside this script when deploying.

```
# Source exported 15 August 2026 from Pakistan Adverse Media Scraper handoff PDF
# File: cnn_bbc_rss.py
# Purpose: internal AML / adverse-media news monitoring utility

#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Pakistan Adverse Media - CNN + BBC RSS (portable single file)
=====
Copy this file AND keywords.xlsx to another PC. Scrapes CNN and BBC for
Pakistan-related adverse-media stories via RSS (native Asia/World feeds +
Google News CNN/BBC search), applies the same keyword lists as scraper.py /
tribune_ocr.py (keywords.xlsx), and saves matching articles as PDFs.

SETUP (once per machine):
    pip install requests beautifulsoup4 reportlab lxml googlenewsdecoder openpyxl

RUN:
    python cnn_bbc_rss.py
        Always prompts for date/time first, e.g. 25-07-2026 09:00
    python cnn_bbc_rss.py --date "25-07-2026 09:00"
    python cnn_bbc_rss.py --no-prompt --date "25-07-2026 09:00"
    python cnn_bbc_rss.py --fresh
    run_int.bat

OUTPUT:
    downloads_int/<Keyword>/<CNN|BBC>/<YYYY-MM-DD>/<title>.pdf
    manifest_int.csv (+ manifest_int_YYYY-MM-DD.csv)

NOTE:
    RSS feeds only keep recent items. A past date finds stories published that
    day that are STILL in the live feed (usually last ~1-7 days), not a full
    archive.
"""

from __future__ import annotations

import argparse
import csv
import hashlib
import re
import shutil
import sys
import time
import xml.etree.ElementTree as ET
from datetime import date, datetime, timedelta, timezone
from email.utils import parsedate_to_datetime
from pathlib import Path
from typing import Dict, List, Optional, Set, Tuple
from urllib.parse import parse_qs, unquote, urlparse

import requests
from bs4 import BeautifulSoup
from reportlab.lib.enums import TA_JUSTIFY, TA_LEFT
from reportlab.lib.pagesizes import LETTER
from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
from reportlab.lib.units import inch
from reportlab.platypus import Paragraph, SimpleDocTemplate, Spacer

try:
    from openpyxl import load_workbook
except ImportError:
    load_workbook = None # type: ignore

BASE = Path(__file__).resolve().parent
OUT = BASE / "downloads_int"
MANIFEST = BASE / "manifest_int.csv"
KEYWORDS_XLSX = BASE / "keywords.xlsx"

MANIFEST_FIELDS = [
    "record_id", "title", "url", "keyword", "all_keywords",
    "source", "date", "pdf_path", "pdf_link", "person_names", "scraped_at",
]

HEADERS = {
    "User-Agent": (
        "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
        "AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36"
    ),
    "Accept": "application/rss+xml, application/xml, text/xml, text/html,*/*;q=0.8",
    "Accept-Language": "en-US,en;q=0.9",
}
```

```
TIMEOUT = 25
DELAY = 0.5
```

```
# Native CNN/BBC feeds (sparse for Pakistan) + Google News scoped to those sites.
```

```
RSS_FEEDS = [
    ("BBC", "https://feeds.bbc.co.uk/news/world/asia/rss.xml"),
    ("BBC", "https://feeds.bbc.co.uk/news/world/rss.xml"),
    ("CNN", "http://rss.cnn.com/rss/edition_asia.rss"),
    ("CNN", "http://rss.cnn.com/rss/edition_world.rss"),
    (
        "GoogleNews",
        "https://news.google.com/rss/search?"
        "q=Pakistan+(terror+OR+terrorism+OR+attack+OR+blast+OR+bomb+OR+"
        "militant+OR+arrested+OR+kidnap+OR+smuggling+OR+FIA+OR+NAB+OR+"
        "corruption+OR+fraud)+&"
        "(site:bbc.com+OR+site:bbc.co.uk+OR+site:cnn.com)"
        "+when:7d&hl=en-US&gl=US&ceid=US:en",
    ),
]
```

```
# -----
# Keywords from shared keywords.xlsx (edit in Excel; restart after changes)
# -----
```

```
def _load_keywords() -> Tuple[Dict[str, List[str]], List[str], Set[str]]:
    if load_workbook is None:
        raise SystemExit("Install openpyxl to read keywords.xlsx: pip install openpyxl")
    path = KEYWORDS_XLSX
    if not path.exists():
        raise SystemExit(
            f"Missing {path.name} next to cnn-bbc-rss.py - copy keywords.xlsx with this script."
        )
    try:
        wb = load_workbook(path, read_only=True, data_only=True)
    except Exception as e:
        raise SystemExit(f"Could not open keywords.xlsx: {e}") from e

    if "keywords" not in wb.sheetnames:
        raise SystemExit("keywords.xlsx must have a sheet named 'keywords'.")

    kws: Dict[str, List[str]] = {}
    seen: Dict[str, Set[str]] = {}
    ws = wb["keywords"]
    rows = ws.iter_rows(values_only=True)
    next(rows, None)
    for row in rows:
        if not row or len(row) < 2:
            continue
        cat, term = row[0], row[1]
        if cat is None or term is None:
            continue
        cat_s, term_s = str(cat).strip(), str(term).strip()
        if not cat_s or not term_s or cat_s.lower() == "category":
            continue
        bucket = seen.setdefault(cat_s, set())
        if term_s.lower() in {t.lower() for t in bucket}:
            continue
        bucket.add(term_s)
        kws.setdefault(cat_s, []).append(term_s)

    pri: List[str] = []
    if "priority" in wb.sheetnames:
        pr = wb["priority"]
        prws = pr.iter_rows(values_only=True)
        next(prws, None)
        for row in prws:
            if not row:
                continue
            cat = row[1] if len(row) > 1 and row[1] is not None else row[0]
            if cat is None:
                continue
            cat_s = str(cat).strip()
            if not cat_s or cat_s.lower() in {"category", "order"}:
                continue
            if cat_s not in pri:
                pri.append(cat_s)

    agency: Set[str] = set()
    if "agency" in wb.sheetnames:
        ag = wb["agency"]
        agrows = ag.iter_rows(values_only=True)
        next(agrows, None)
        for row in agrows:
            if not row or row[0] is None:
                continue
            cat_s = str(row[0]).strip()
            if cat_s and cat_s.lower() != "category":
                agency.add(cat_s)

    wb.close()

    if not kws:
        raise SystemExit("keywords.xlsx sheet 'keywords' has no category/term rows.")
    if not pri:
```

```

        pri = list(kws.keys())
    if not agency:
        agency = {"ANF", "CTD", "Customs", "FIA", "NAB", "NCCIA"}
    return kws, pri, agency

KEYWORDS, PRIORITY, AGENCY_KEYWORDS = _load_keywords()

CRIME_ACTION = re.compile(
    r"\b(?:"
    r"arrest(?:ed|s)?|apprehend(?:ed|s)?|detain(?:ed|s)?|remand|"
    r"raid(?:ed|s)?|crackdown|bust(?:ed|s)?|foils?|foiled|"
    r"seiz(?:ed|ure|es)|confiscat(?:ed|ion)|recover(?:ed|y)|"
    r"charg(?:ed|es)|convict(?:ed|ion)|sentenc(?:ed|e)|indict(?:ed|ment)|"
    r"FIIR|accused|suspect|"
    r"kill(?:ed|s)?|martyred|shot\s+dead|gunned\s+down|assassinated|"
    r"injur(?:ed|y|ies)|wounded|"
    r"attack(?:ed|s)?|ambush(?:ed)?|blast|bomb(?:ing|ed)?|grenade|explosion|ied|"
    r"firing|gunfire|encounter(?:s)?|"
    r"kidnap(?:ped|ping)?|abduct(?:ed|ion)?|ransom|"
    r"smuggl(?:ed|ing|er)|traffick(?:ed|ing|er)|"
    r"extort(?:ion|ed)?|brib(?:e|ery|ed)|embezzl(?:e|ed|ement)|"
    r"fraud(?:ulent)?|scam|launder(?:ing|ed)?|"
    r"narcotics?|drug\s+(?:peddler|dealer|den)|"
    r"terror(?:ism|ist)?\s+attack|militant|"
    r"held|nabbed"
    r")\b",
    re.I,
)

ATTACK_VERB = re.compile(
    r"\b(?:"
    r"attack|ambush|blast|bomb|grenade|explosion|ied|firing|gunfire|"
    r"militant|terror|drone|encounter(?:s)?|martyred|shot\s+dead|gunned\s+down"
    r")\b",
    re.I,
)

# Strong Pakistan markers - prefer these in title/summary.
PK_STRONG = re.compile(
    r"\b(?:"
    r"Pakistan|Pakistani|Islamabad|Karachi|Lahore|Rawalpindi|Peshawar|Quetta|"
    r"Multan|Faisalabad|Hyderabad|Gwadar|Balochistan|Sindh|"
    r"Khyber|K-P\b|Waziristan|Tank|Kohat|Bannu|Bajaur|Chaman|Taftan|Turbat|"
    r"Azad\s+Kashmir|Pakistan[- ]administered\s+Kashmir|"
    r"FIA|NAB|ANF|CTD|NCCIA|TTP|Tehreek[- ]e[- ]Taliban|"
    r"Imran\s+Khan"
    r")\b",
    re.I,
)

# Weaker markers that alone are NOT enough (Punjab also means India).
PK_WEAK = re.compile(r"\b(?:Punjab|Kashmir)\b", re.I)

EXCLUDE = re.compile(
    r"(?:"
    r"\bcricket\b|\bfootball\b|\btest\s+match\b|\bbowling\b|\bwicket|"
    r"commonwealth\s+games|premier\s+league|world\s+cup|"
    r"bollywood|hollywood|movie|celebrity|fashion|"
    r"brain(?:s|fall)?\b|\bflood(?:ed|ing|s)?\b|monsoon|torrential|"
    r"collision|accident|generator\s+smoke|"
    r"bhails?\b|\binaugurat|"
    r"stock\s+market|KSE-100|"
    r"fast\s+facts|" # CNN stub pages
    r"racial\s+discrimination|australian\s+court" # false PK hits via title noise
    r")",
    re.I,
)

CNN_BBC_HOST = re.compile(
    r"(?:^|\.)(?:bbc\.com|bbc\.co\.uk|cnn\.com)$", re.I
)

session = requests.Session()
session.headers.update(HEADERS)

def _compile_patterns() -> dict:
    out = {}
    for folder, terms in KEYWORDS.items():
        escaped = [re.escape(t) for t in sorted(terms, key=len, reverse=True)]
        out[folder] = re.compile(r"\b(?:" + "|".join(escaped) + r")\b", re.I)
    return out

PATTERNS = _compile_patterns()

def find_keywords(title: str) -> List[str]:
    hits = []
    for folder, pat in PATTERNS.items():
        if pat.search(title or ""):
```

```

        hits.append(folder)
    return hits

def primary_kw(kws: List[str]) -> str:
    for p in PRIORITY:
        if p in kws:
            return p
    return kws[0] if kws else "General"

def is_pakistan_story(title: str, summary: str = "", body: str = "") -> bool:
    """Require a strong Pakistan marker in title/summary (or early body)."""
    head = f"{title} {summary}"
    if PK_STRONG.search(head):
        return True
    # Weak markers (Punjab/Kashmir) only count with Pakistan context nearby
    if PK_WEAK.search(head) and re.search(
        r"\b(?:Pakistan|Pakistani|Islamabad|administered)\b", head, re.I
    ):
        return True
    # Body fallback for short RSS titles - first 800 chars only
    if PK_STRONG.search((body or "")[:800]):
        return True
    return False

def is_relevant(
    title: str, body: str = "", summary: str = ""
) -> Tuple[bool, List[str], str]:
    if not is_pakistan_story(title, summary, body):
        return False, [], "not_pakistan"

    if EXCLUDE.search(title) or EXCLUDE.search(f"{title} {summary}[:400]"):
        return False, [], "excluded_topic"

    kws = find_keywords(title)
    if not kws:
        # Short wire titles: allow one keyword from summary line
        kws = find_keywords(summary or "")
    if not kws:
        return False, [], "no_keyword"

    combined = f"{title} {summary} {body}"
    if "Attack" in kws and not ATTACK_VERB.search(title):
        if not ATTACK_VERB.search(combined[:500]):
            return False, [], "attack_without_verb"

    agency_hits = [k for k in kws if k in AGENCY_KEYWORDS]
    non_agency = [k for k in kws if k not in AGENCY_KEYWORDS]
    if agency_hits and not non_agency:
        if not CRIME_ACTION.search(title) and not CRIME_ACTION.search(summary or ""):
            return False, [], "agency_without_action"

    if not CRIME_ACTION.search(title) and not CRIME_ACTION.search(combined[:1000]):
        return False, [], "no_crime_action"

    if agency_hits and non_agency:
        final = sorted(set(non_agency + agency_hits))
    elif agency_hits:
        final = agency_hits
    else:
        final = non_agency
    return True, sorted(set(final)), "ok"

def story_id(title: str) -> str:
    t = re.sub(r"^[^w\s]", " ", (title or "").lower())
    t = re.sub(r"\s+", " ", t).strip()
    return hashlib.md5(t.encode()).hexdigest()[:12]

def safe_name(title: str) -> str:
    t = re.sub(r'[\&\&\&:"/\|?]*', "", title)
    t = re.sub(r"\s+", "_", t.strip())[:80]
    return t or "article"

# -----
# Person-name extraction (same idea as name_extractor.py; no spaCy required)
# -----

STOP_TOKENS = {
    "Pakistan", "Karachi", "Lahore", "Islamabad", "Peshawar", "Quetta",
    "Sindh", "Punjab", "Balochistan", "Khyber", "Afghanistan", "India",
    "China", "Iran", "Iraq", "Kashmir", "BBC", "CNN", "Reuters",
    "Thursday", "Friday", "Saturday", "Sunday", "Monday", "Tuesday",
    "Wednesday", "January", "February", "March", "April", "May", "June",
    "July", "August", "September", "October", "November", "December",
    "Court", "Police", "Agency", "Minister", "Prime", "Chief", "Justice",
    "High", "Supreme", "National", "Federal", "Investigation", "Force",
    "Army", "Navy", "World", "Bank", "United", "States",
    "FIA", "NAB", "ANF", "CTD", "NCCIA", "TTP", "IGP", "DG",

```



```

    "The", "According", "After", "Although", "During", "Earlier",
    "Following", "However", "Later", "Meanwhile", "Officials", "Report",
    "Sources", "Source", "Former", "Private", "Public", "Foreign",
    "Pakistani", "British", "American", "Indian", "Iranian", "Afghan",
    "Accused", "Suspect", "Suspects", "Arrested", "Arrest", "Raid",
    "Man", "Woman", "Four", "Three", "Two", "One", "All",
}

TITLE_PREFIXES = re.compile(
    r"\b(?:Mr|Mrs|Ms|Miss|Dr|Prof|Justice|Barrister|Advocate|"
    r"Senator|MNA|MPA|CM|PM|IGP|DG|SSP|DSP|SHO)\.?\s+",
    re.IGNORECASE,
)

IDENTIFIED_AS = re.compile(
    r"(?:identified as|named as|arrested(?:\s\w+){0,3}\s+as|"
    r"apprehended(?:\s\w+){0,3}\s+as|accused)\s+"
    r"([A-Z][a-z]+(?:\s+[A-Z][a-z]+){0,3})",
    re.IGNORECASE,
)

CAPITALIZED_NAME = re.compile(
    r"\b([A-Z][a-z]{2,}(?:\s+[A-Z][a-z]{2,}){0,3})\b"
)

SAID_PATTERN = re.compile(
    r"\b([A-Z][a-z]+(?:\s+[A-Z][a-z]+){0,3})\s+"
    r"(?:said|told|stated|added|claimed|alleged|confirmed|informed|disclosed)\b"
)

def _clean_person_name(name: str) -> str:
    name = re.sub(r"\s+", " ", name.strip())
    return re.sub(r"[,;]+$", "", name)

def _is_valid_person_name(name: str) -> bool:
    if not name or len(name) < 3:
        return False
    parts = name.split()
    if len(parts) > 4:
        return False
    for part in parts:
        if part in STOP_TOKENS or len(part) < 2 or not part[0].isupper():
            return False
    return True

def extract_person_names(text: str) -> List[str]:
    """Extract likely person names from title + body for the manifest column."""
    if not text:
        return []
    names: Set[str] = set()
    for match in IDENTIFIED_AS.finditer(text):
        candidate = _clean_person_name(match.group(1))
        if _is_valid_person_name(candidate):
            names.add(candidate)
    for match in TITLE_PREFIXES.finditer(text):
        snippet = text[match.end() : match.end() + 60]
        name_match = CAPITALIZED_NAME.match(snippet)
        if name_match:
            candidate = _clean_person_name(name_match.group(1))
            if _is_valid_person_name(candidate):
                names.add(candidate)
    for match in SAID_PATTERN.finditer(text):
        candidate = _clean_person_name(match.group(1))
        if _is_valid_person_name(candidate):
            names.add(candidate)
    return sorted(names)

def strip_source_suffix(title: str) -> str:
    """Google News titles end with ' - BBC' / ' - CNN'."""
    return re.sub(
        r"\s*[-|]\s*(?:BBC(?:\s+News)?|CNN(?:\.com)?)\s*$",
        "",
        title or "",
        flags=re.I,
    ).strip()

def detect_source(url: str, feed_source: str, title: str = "") -> str:
    host = urlparse(url).netloc.lower()
    if "bbc." in host or re.search(r"\bBBC\b", title or ""):
        return "BBC"
    if "cnn." in host or re.search(r"\bCNN\b", title or ""):
        return "CNN"
    if feed_source in ("BBC", "CNN"):
        return feed_source
    return "Unknown"

def is_cnn_or_bbc_url(url: str) -> bool:
    host = urlparse(url).netloc.lower()
    host = host[4:] if host.startswith("www.") else host
    return bool(CNN_BBC_HOST.search(host))

```

```

# -----
# HTTP / RSS parsing
# -----

def get_text(url: str) -> Optional[str]:
    try:
        r = session.get(url, timeout=TIMEOUT, allow_redirects=True)
        r.raise_for_status()
        r.encoding = r.apparent_encoding or "utf-8"
        time.sleep(DELAY)
        return r.text
    except Exception as e:
        print(f" [WARN] {url}: {e}")
        return None

def get_soup(url: str) -> Optional[BeautifulSoup]:
    text = get_text(url)
    if not text:
        return None
    return BeautifulSoup(text, "html.parser")

def resolve_google_news_url(url: str) -> str:
    """Follow / decode Google News article links to the publisher URL."""
    if "news.google.com" not in url:
        return url

    # Preferred: googlenewsdecoder (Google no longer redirects cleanly)
    try:
        from googlenewsdecoder import gnewsdecoder

        result = gnewsdecoder(url)
        if isinstance(result, dict) and result.get("status") and result.get("decoded_url"):
            decoded = result["decoded_url"]
            if is_cnn_or_bbc_url(decoded):
                return decoded
            return decoded
    except Exception:
        pass

    try:
        r = session.get(url, timeout=TIMEOUT, allow_redirects=True)
        time.sleep(DELAY)
        final = r.url
        if is_cnn_or_bbc_url(final):
            return final
        m = re.search(
            r"href=['\"](https?:/{0}?(www\.)?(?:bbc\.com|bbc\.co\.uk|cnn\.com))[^'\"]+)",
            r.text,
            re.I,
        )
        if m:
            return m.group(1)
        m = re.search(r"[?&]url=(^&.+)", url)
        if m:
            return unquote(m.group(1))
        return final
    except Exception as e:
        print(f" [WARN] resolve {url}: {e}")
        return url

def _local_tag(tag: str) -> str:
    return tag.rsplit("}", 1)[-1] if "}" in tag else tag

def _child_text(el: ET.Element, names: Tuple[str, ...]) -> str:
    for child in list(el):
        if _local_tag(child.tag).lower() in names:
            return (child.text or "").strip()
    return ""

def _child_link(el: ET.Element) -> str:
    for child in list(el):
        if _local_tag(child.tag).lower() != "link":
            continue
        href = child.attrib.get("href") or child.attrib.get("url")
        if href:
            return href.strip()
        if child.text and child.text.strip():
            return child.text.strip()
    for child in list(el):
        if _local_tag(child.tag).lower() == "guid":
            text = (child.text or "").strip()
            if text.startswith("http"):
                return text
    return ""

def parse_pub_date(raw: str) -> Optional[date]:
    if not raw:

```

```

        return None
    raw = raw.strip()
    try:
        dt = parsedate_to_datetime(raw)
        if dt.tzinfo is None:
            dt = dt.replace(tzinfo=timezone.utc)
        return dt.astimezone().date()
    except Exception:
        pass
    m = re.search(r"(\d{4})-(\d{2})-(\d{2})", raw)
    if m:
        return date(int(m.group(1)), int(m.group(2)), int(m.group(3)))
    return None

def parse_rss_items(xml_text: str, feed_source: str) -> List[dict]:
    items: List[dict] = []
    try:
        root = ET.fromstring(xml_text)
    except ET.ParseError as e:
        print(f" [WARN] Bad RSS XML ({feed_source}): {e}")
        return items

    nodes = [
        el for el in root.iter()
        if _local_tag(el.tag).lower() in ("item", "entry")
    ]
    for el in nodes:
        title = strip_source_suffix(_child_text(el, ("title",)))
        link = _child_link(el)
        summary = _child_text(el, ("description", "summary", "content"))
        summary = BeautifulSoup(summary, "html.parser").get_text(" ", strip=True)
        pub = _child_text(el, ("pubdate", "published", "updated", "date"))
        pub_day = parse_pub_date(pub)
        if not title or not link:
            continue
        items.append({
            "title": title.strip(),
            "url": link.strip(),
            "feed_source": feed_source,
            "summary": summary,
            "pub_date": pub_day.isoformat() if pub_day else "",
        })
    return items

def extract_article_body(url: str) -> str:
    soup = get_soup(url)
    if not soup:
        return ""
    host = urlparse(url).netloc.lower()
    if "bbc." in host:
        selectors = [
            "article",
            "[data-component='text-block']",
            "main",
        ]
    elif "cnn." in host:
        selectors = [
            ".article__content",
            ".article-content",
            "[class*='article__content']",
            "article",
            "main",
        ]
    else:
        selectors = ["article", "main", ".story-body", ".article-body"]

    chunks: List[str] = []
    for sel in selectors:
        for el in soup.select(sel):
            text = el.get_text(" ", strip=True)
            if len(text) > 120:
                chunks.append(text)
        if chunks:
            break
    if not chunks:
        meta = soup.find("meta", attrs={"name": "description"})
        if meta and meta.get("content"):
            return meta["content"]
        return ""
    return max(chunks, key=len)[:12000]

def save_pdf(
    path: Path,
    title: str,
    source: str,
    url: str,
    date_str: str,
    body: str,
    kws: List[str],
    person_names: Optional[List[str]] = None,

```

```

) -&gt; None:
    path.parent.mkdir(parents=True, exist_ok=True)
    styles = getSampleStyleSheet()
    title_s = ParagraphStyle(
        "T", parent=styles["Heading1"], fontSize=14, leading=18,
        alignment=TA_LEFT, spaceAfter=10,
    )
    meta_s = ParagraphStyle(
        "M", parent=styles["Normal"], fontSize=9, textColor="#444", spaceAfter=4,
    )
    body_s = ParagraphStyle(
        "B", parent=styles["Normal"], fontSize=11, leading=15,
        alignment=TA_JUSTIFY, spaceAfter=6,
    )

def esc(s: str) -&gt; str:
    return (s or "").replace("&","&").replace("<","&lt;").replace(">","&gt;")

names = person_names or []
story = [
    Paragraph(esc(title), title_s),
    Spacer(1, 0.08 * inch),
    Paragraph(f"<b>Source:</b> {esc(source)}", meta_s),
    Paragraph(f"<b>Date:</b> {esc(date_str)}", meta_s),
    Paragraph(f"<b>URL:</b> {esc(url)}", meta_s),
    Paragraph(f"<b>Keywords:</b> {esc(' '.join(kws))}", meta_s),
    Paragraph(
        f"<b>Person Names:</b> {esc(' '.join(names) if names else 'None identified')}",
        meta_s,
    ),
    Spacer(1, 0.15 * inch),
]
for para in re.split(r"(?<=[.!?])\s+(?=[A-Z])", (body or "").strip()) or [body]:
    if para and para.strip():
        story.append(Paragraph(esc(para.strip()), body_s))
SimpleDocTemplate(
    str(path), pagesize=LETTER,
    rightMargin=50, leftMargin=50, topMargin=50, bottomMargin=50,
).build(story)

def load_seen() -&gt; Set[str]:
    seen: Set[str] = set()
    for p in [MANIFEST, *BASE.glob("manifest_int*.csv")]:
        if not p.exists():
            continue
        try:
            with p.open(encoding="utf-8", newline="") as f:
                for row in csv.DictReader(f):
                    if row.get("title"):
                        seen.add(story_id(row["title"]))
        except OSError:
            pass
    return seen

def clear_manifests() -&gt; None:
    for p in [
        MANIFEST,
        MANIFEST.with_suffix(".xlsx"),
        *BASE.glob("manifest_int*.csv"),
        *BASE.glob("manifest_int*.xlsx"),
    ]:
        try:
            if p.exists():
                p.unlink()
        except OSError:
            pass

def pdf_hyperlink(pdf: Path) -&gt; str:
    uri = pdf.resolve().as_uri().replace("'", '"')
    return f'=HYPERLINK("{uri}","Open PDF")'

def resolve_pdf_from_manifest(pdf_path_val: str) -&gt; Optional[Path]:
    if not pdf_path_val:
        return None
    raw = pdf_path_val.strip().strip('"')
    m = re.search(r'HYPERLINK("(^")+)"', raw, re.I)
    if m:
        raw = m.group(1)
    if raw.lower().startswith("file:"):
        from urllib.parse import urlparse, unquote
        import os
        parsed = urlparse(raw)
        path = unquote(parsed.path)
        if os.name == "nt" and path.startswith("/") and len(path) >= 3 and path[2] == ":":
            path = path[1:]
        return Path(path)
    p = Path(raw)
    if p.is_absolute():
        return p

```

```

cand = OUT / p
return cand if cand.exists() else (BASE / p)

def sync_manifest_xlsx(csv_path: Path) -> None:
    if load_workbook is None or not csv_path.exists():
        return
    try:
        from openpyxl import Workbook
        from openpyxl.styles import Font
    except ImportError:
        return
    try:
        with csv_path.open(encoding="utf-8", newline="") as f:
            rows = list(csv.DictReader(f))
            fields = list(rows[0].keys()) if rows else list(MANIFEST_FIELDS)
    except OSError:
        return
    if "pdf_link" not in fields:
        if "pdf_path" in fields:
            i = fields.index("pdf_path") + 1
            fields = fields[:i] + ["pdf_link"] + [c for c in fields[i:] if c != "pdf_link"]
        else:
            fields = fields + ["pdf_link"]
    wb = Workbook()
    ws = wb.active
    ws.title = "manifest"
    ws.append(fields)
    link_font = Font(color="0563C1", underline="single")
    for row in rows:
        values = [(row.get(c, "") if c != "pdf_link" else "") for c in fields]
        ws.append(values)
        r_idx = ws.max_row
        pdf = resolve_pdf_from_manifest(row.get("pdf_path", "") or "")
        if pdf is None:
            pdf = resolve_pdf_from_manifest(row.get("pdf_link", "") or "")
        cell = ws.cell(row=r_idx, column=fields.index("pdf_link") + 1)
        if pdf and pdf.exists():
            cell.hyperlink = pdf.resolve().as_uri()
            cell.value = "Open PDF"
            cell.font = link_font
        else:
            cell.value = row.get("pdf_link", "") or ""
    try:
        wb.save(csv_path.with_suffix(".xlsx"))
    except OSError:
        pass

def ensure_manifest_schema(path: Path) -> None:
    if not path.exists():
        return
    try:
        with path.open(encoding="utf-8", newline="") as f:
            reader = csv.DictReader(f)
            old_fields = list(reader.fieldnames or [])
            rows = list(reader)
    except OSError:
        return
    needs_rewrite = old_fields != MANIFEST_FIELDS
    upgraded = []
    for row in rows:
        pdf_path = row.get("pdf_path", "") or ""
        pdf_link = (row.get("pdf_link") or "").strip()
        if not pdf_link and pdf_path:
            pdf = resolve_pdf_from_manifest(pdf_path)
            if pdf:
                pdf_link = pdf.hyperlink(pdf)
                needs_rewrite = True
        if "pdf_link" not in old_fields:
            needs_rewrite = True
        upgraded.append({k: (pdf_link if k == "pdf_link" else row.get(k, "")) for k in MANIFEST_FIELDS})
    if not needs_rewrite:
        sync_manifest_xlsx(path)
        return
    try:
        tmp = path.with_suffix(path.suffix + ".tmp")
        with tmp.open("w", encoding="utf-8", newline="") as f:
            w = csv.DictWriter(f, fieldnames=MANIFEST_FIELDS)
            w.writeheader()
            w.writerows(upgraded)
        tmp.replace(path)
        sync_manifest_xlsx(path)
    except OSError:
        return

def append_manifest(row: dict) -> None:
    payload = {k: row.get(k, "") for k in MANIFEST_FIELDS}
    for path in [MANIFEST, BASE / f"manifest_int_{row['date']}.csv"]:
        try:
            ensure_manifest_schema(path)
            exists = path.exists()

```

```

        with path.open("a", encoding="utf-8", newline="") as f:
            w = csv.DictWriter(f, fieldnames=MANIFEST_FIELDS)
            if not exists:
                w.writeheader()
            w.writerow(payload)
        sync_manifest_xlsx(path)
    except PermissionError:
        continue

# -----
# Date / time prompt
# -----

WEEKDAY_NAMES = {
    "monday": 0, "mon": 0,
    "tuesday": 1, "tue": 1, "tues": 1,
    "wednesday": 2, "wed": 2,
    "thursday": 3, "thu": 3, "thur": 3, "thurs": 3,
    "friday": 4, "fri": 4,
    "saturday": 5, "sat": 5,
    "sunday": 6, "sun": 6,
}

def parse_datetime_input(raw: str) -> Tuple[str, str]:
    today = date.today()
    raw = (raw or "").strip()
    if not raw:
        now = datetime.now()
        return today.isoformat(), now.strftime("%Y-%m-%d %H:%M:%S")

    m = re.match(
        r"^(\d{2})-(\d{2})-(\d{4})"
        r"(?:\s+(\d{1,2}):(\d{2}))?(?:\s+(\d{2}))??"$,
        raw,
    )
    if m:
        d, mo, y = int(m.group(1)), int(m.group(2)), int(m.group(3))
        edition = date(y, mo, d)
        if m.group(4) is not None:
            hh, mm = int(m.group(4)), int(m.group(5))
            ss = int(m.group(6) or 0)
            run_dt = datetime(y, mo, d, hh, mm, ss)
        else:
            now = datetime.now()
            run_dt = datetime(y, mo, d, now.hour, now.minute, now.second)
        return edition.isoformat(), run_dt.strftime("%Y-%m-%d %H:%M:%S")

    m = re.match(
        r"^(\d{4})-(\d{2})-(\d{2})"
        r"(?:\s+(\d{1,2}):(\d{2}))?(?:\s+(\d{2}))??"$,
        raw,
    )
    if m:
        y, mo, d = int(m.group(1)), int(m.group(2)), int(m.group(3))
        edition = date(y, mo, d)
        if m.group(4) is not None:
            hh, mm = int(m.group(4)), int(m.group(5))
            ss = int(m.group(6) or 0)
            run_dt = datetime(y, mo, d, hh, mm, ss)
        else:
            now = datetime.now()
            run_dt = datetime(y, mo, d, now.hour, now.minute, now.second)
        return edition.isoformat(), run_dt.strftime("%Y-%m-%d %H:%M:%S")

    low = raw.lower()
    if low == "today":
        now = datetime.now()
        return today.isoformat(), now.strftime("%Y-%m-%d %H:%M:%S")
    if low == "yesterday":
        now = datetime.now()
        ed = today - timedelta(days=1)
        return ed.isoformat(), now.strftime("%Y-%m-%d %H:%M:%S")
    if low in WEEKDAY_NAMES:
        target_wd = WEEKDAY_NAMES[low]
        delta = (today.weekday() - target_wd) % 7
        ed = today - timedelta(days=delta)
        now = datetime.now()
        return ed.isoformat(), now.strftime("%Y-%m-%d %H:%M:%S")

    raise ValueError(
        f"Invalid date/time: {raw!r}\n"
        f"Use: 25-07-2026 09:00 (DD-MM-YYYY HH:MM)"
    )

def ask_datetime(default: str = "") -> Tuple[str, str]:
    """Always ask for one date+time before scraping starts (same as scraper.py)."""
    today = date.today()
    print()
    print("=" * 50)
    print("Pakistan Adverse Media - CNN + BBC RSS")

```

```

print("=" * 50)
print(f"Today is {today.strftime('%A')} {today.strftime('%d-%m-%Y')}")
print()
print("Enter ONE edition date and time, then press Enter.")
print("  Format:  DD-MM-YYYY HH:MM")
print("  Example: 25-07-2026 09:00")
print()
print("When finished, run again and type the NEXT date/time.")
print("(Saturday, then Sunday, then Monday on a Monday morning.)")
print()
if default:
    print(f"  (Press Enter to use: {default})")
sys.stdout.flush()
try:
    raw = input("Date and time> ").strip()
except EOFError:
    raw = ""
if not raw:
    raw = default
if not raw:
    print("No date entered. Using today.")
    return parse_datetime_input("")
return parse_datetime_input(raw)

# -----
# Collect + process
# -----

def collect_rss_articles(target: str) -> List[dict]:
    seen_urls: Set[str] = set()
    seen_titles: Set[str] = set()
    collected: List[dict] = []

    for feed_source, feed_url in RSS_FEEDS:
        label = "Google News (CNN/BBC)" if feed_source == "GoogleNews" else feed_source
        print(f"=== {label} RSS ===")
        print(f"  Feed: {feed_url[:90]}{'...' if len(feed_url) > 90 else ''}")
        xml_text = get_text(feed_url)
        if not xml_text:
            continue
        items = parse_rss_items(xml_text, feed_source)
        pk_items = [
            it for it in items
            if is_pakistan_story(it["title"], it.get("summary", ""))
        ]
        print(f"  Items: {len(items)} | Pakistan-related: {len(pk_items)}")

        for it in pk_items:
            if it.get("pub_date") and it["pub_date"] != target:
                continue
            url = it["url"]
            if "news.google.com" in url:
                url = resolve_google_news_url(url)
            if not is_cnn_or_bbc_url(url) and feed_source == "GoogleNews":
                continue
            if url in seen_urls:
                continue
            tid = story_id(it["title"])
            if tid in seen_titles:
                continue
            seen_urls.add(url)
            seen_titles.add(tid)
            source = detect_source(url, feed_source, it["title"])
            if source == "Unknown":
                continue
            collected.append({
                **it,
                "url": url,
                "source": source,
            })

    print(f"\nEnriching {len(collected)} Pakistan CNN/BBC items for {target}...")
    arts: List[dict] = []
    for i, it in enumerate(collected, 1):
        if i % 5 == 0 or i == 1:
            print(f"  Enrich {i}/{len(collected)}...")
        body = extract_article_body(it["url"])
        if not is_pakistan_story(it["title"], it.get("summary", ""), body):
            continue
        date_str = it.get("pub_date") or target
        if date_str != target:
            continue
        arts.append({
            "title": it["title"],
            "url": it["url"],
            "source": it["source"],
            "date": date_str,
            "body": body or it.get("summary", ""),
            "summary": it.get("summary", ""),
        })
    return arts

def process_articles(

```

```

    articles: List[dict], seen: Set[str], stats: dict, run_stamp: str, target: str
) -> None:
    for art in articles:
        art["date"] = target
        stats["checked"] += 1
        ok, kws, reason = is_relevant(
            art["title"], art.get("body", ""), art.get("summary", "")
        )
        if not ok:
            if reason != "no_keyword":
                stats["filtered"] += 1
                print(f" [FILTER] {reason} | {art['source']} | {art['title'][:55]}")
                continue
            stats["matched"] += 1
            sid = story_id(art["title"])
            if sid in seen:
                stats["dup"] += 1
                print(f" [SKIP] dup | {art['source']} | {art['title'][:50]}")
                continue
            pk = primary_kw(kws)
            body_text = art.get("body", "") or art.get("summary", "")
            person_names = extract_person_names(f"{art['title']} {body_text}")
            pdf = OUT / pk / art["source"] / art["date"] / f"{safe_name(art['title'])}.pdf"
            save_pdf(
                pdf, art["title"], art["source"], art["url"], art["date"],
                body_text, kws, person_names=person_names,
            )
            append_manifest({
                "record_id": sid,
                "title": art["title"],
                "url": art["url"],
                "keyword": pk,
                "all_keywords": "; ".join(kws),
                "source": art["source"],
                "date": art["date"],
                "pdf_path": str(pdf.relative_to(OUT)),
                "pdf_link": pdf_hyperlink(pdf),
                "person_names": "; ".join(person_names),
                "scraped_at": run_stamp,
            })
            seen.add(sid)
            stats["saved"] += 1
            print(f" [SAVED] {pk} | {art['source']} | {art['title'][:55]}")
            if person_names:
                print(f"          Names: {' '.join(person_names[:8])}")

def main() -> int:
    ap = argparse.ArgumentParser(
        description="Pakistan adverse media from CNN/BBC RSS (portable)"
    )
    ap.add_argument(
        "--date",
        default="",
        help='Optional default at prompt, e.g. "25-07-2026 09:00"',
    )
    ap.add_argument(
        "--no-prompt",
        action="store_true",
        help="Skip prompt; requires --date",
    )
    ap.add_argument(
        "--fresh",
        action="store_true",
        help="Clear downloads_int + manifest_int*.csv",
    )
    args = ap.parse_args()

    try:
        if args.no_prompt:
            if not args.date:
                print('With --no-prompt you must pass --date "DD-MM-YYYY HH:MM"')
                return 1
            target, run_stamp = parse_datetime_input(args.date)
        else:
            target, run_stamp = ask_datetime(default=args.date or "")
    except ValueError as e:
        print(e)
        return 1
    except (EOFError, KeyboardInterrupt):
        print("\nCancelled.")
        return 1

    if args.fresh:
        if OUT.exists():
            shutil.rmtree(OUT)
        clear_manifests()
        print(f"[fresh] Cleared downloads_int/ and manifest_int*.csv")

    OUT.mkdir(parents=True, exist_ok=True)
    seen = load_seen()
    if seen:
        print(

```



```

        f"Duplicate memory: {len(seen)} titles from manifest_int*.csv "
        f"(deleting downloads_int alone does NOT clear this)"
    )

print()
print(f"Target date: {target}")
print(f"Start time: {run_stamp}")
print(f"Output: {OUT}")
print("-" * 50)

stats = {"checked": 0, "matched": 0, "saved": 0, "dup": 0, "filtered": 0}

try:
    articles = collect_rss_articles(target)
except Exception as e:
    print(f" [ERROR] RSS collect: {e}")
    articles = []

print(f"\nCandidates after enrich: {len(articles)}")
process_articles(articles, seen, stats, run_stamp, target)

print("\n" + "=" * 50)
print("DONE (CNN + BBC RSS)")
print(f" Date: {target}")
print(f" Started: {run_stamp}")
print(f" Checked: {stats['checked']}")
print(f" Keywords: {stats['matched']}")
print(f" Filtered: {stats['filtered']}")
print(f" Saved: {stats['saved']}")
print(f" Dupes: {stats['dup']}")
print(f" Manifest: {MANIFEST}")
print("=" * 50)
print()
print("Re-run and enter the NEXT date/time to continue.")
return 0

if __name__ == "__main__":
    sys.exit(main())

```

scraper_int.py

Thin launcher that calls `cnn_bbc_rss.main()`.

Full source listing - copy into a file named `scraper_int.py`. Encoding: UTF-8. Keep `keywords.xlsx` beside this script when deploying.

```
# Source exported 15 August 2026 from Pakistan Adverse Media Scraper handoff PDF
# File: scraper_int.py
# Purpose: internal AML / adverse-media news monitoring utility

#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""Compatibility shim - portable program lives in cnn_bbc_rss.py."""

from cnn_bbc_rss import main

if __name__ == "__main__":
    raise SystemExit(main())
```

End of handoff package

Thank you for reviewing this internal tooling package. No further action is required unless you are deploying or modifying the scrapers. Always run only on approved systems and follow your organisation's data-handling policy for news archives and manifests.

Generated automatically from the project workspace on 15 August 2026.